

Parallel Programming Patterns

Moreno Marzolla
Dip. di Informatica—Scienza e Ingegneria (DISI)
Università di Bologna

<https://www.moreno.marzolla.name/>

Copyright © 2013, 2017–2020, 2024, 2025
Moreno Marzolla, Università di Bologna, Italy
<https://www.moreno.marzolla.name/teaching/APAI/>



This work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0). To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

What is a *pattern*?

- A design pattern is “*a general solution to a recurring engineering problem*”
- A design pattern is **not** a ready-made solution to a given problem...
- ...rather, it is a description of how a certain **kind** of problem can be solved

Architectural patterns

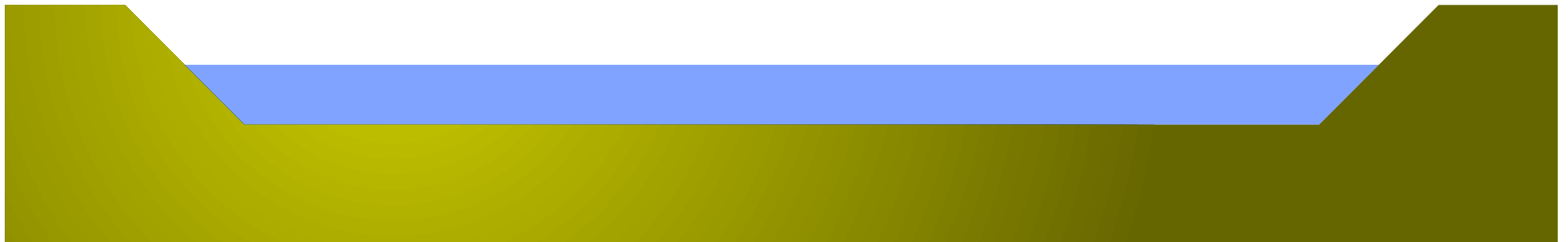
- The term “*architectural pattern*” was first used by architect Christopher Alexander to denote common design decision that have been used by architects and engineers to realize buildings and constructions in general



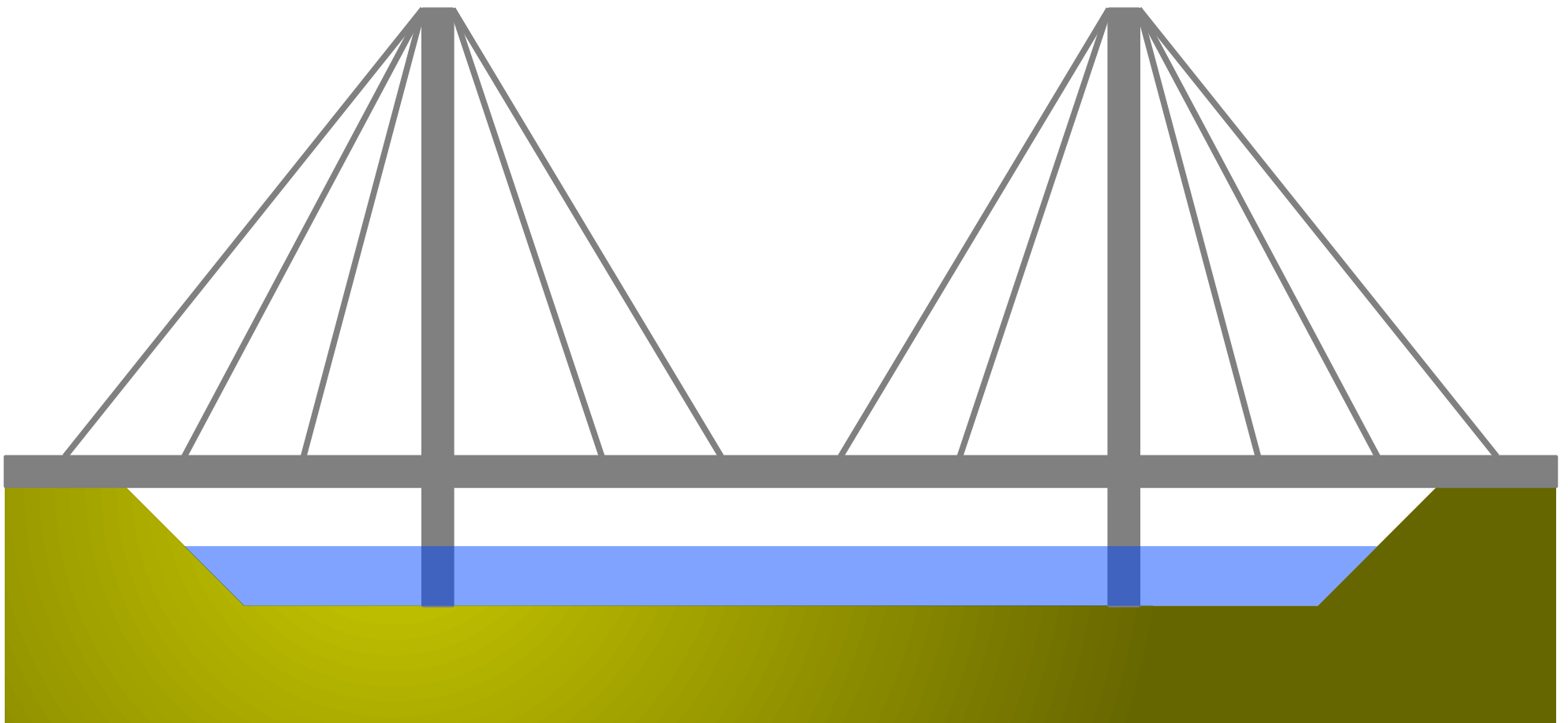
Christopher Alexander,
(1936--), *A Pattern Language:
Towns, Buildings, Construction*

Example

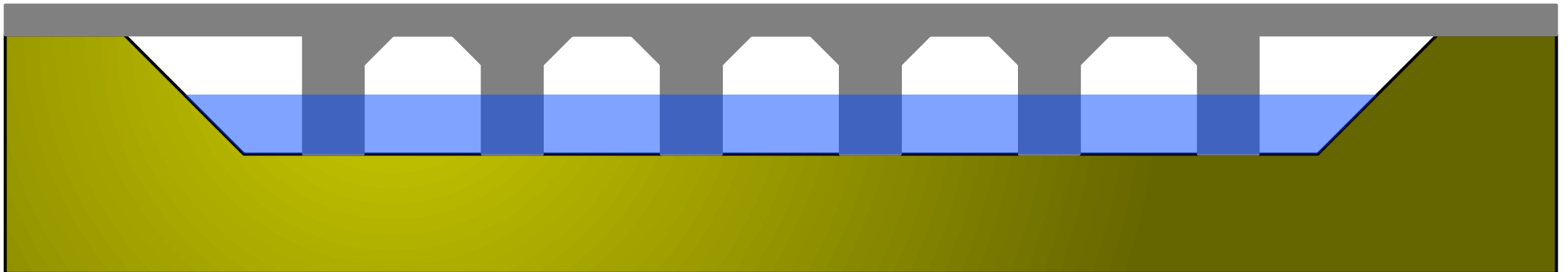
- Building a bridge across a river
- You do not “invent” a brand new type of bridge each time
 - Instead, you adapt an already existing type of bridge



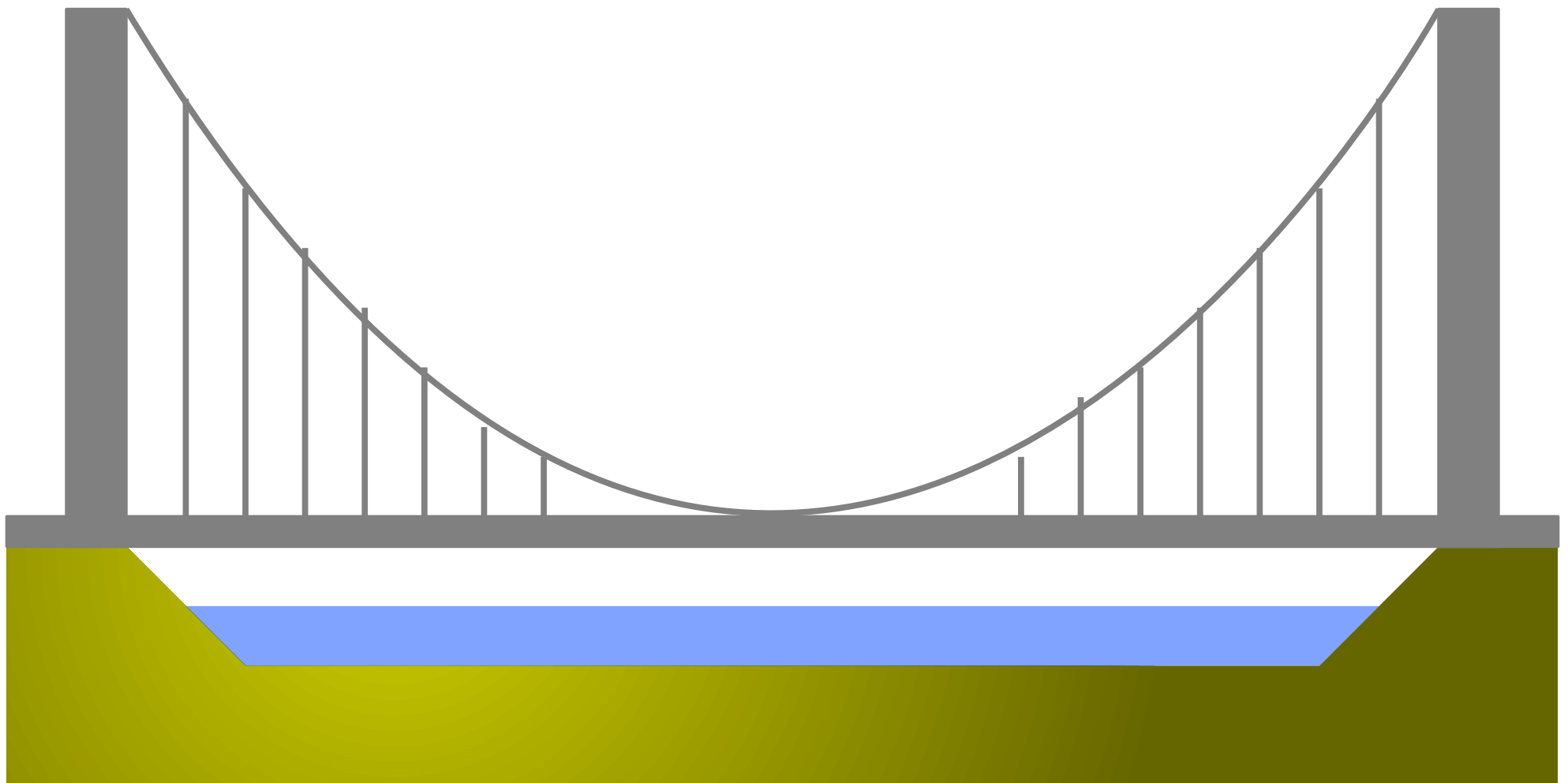
Example



Example



Example

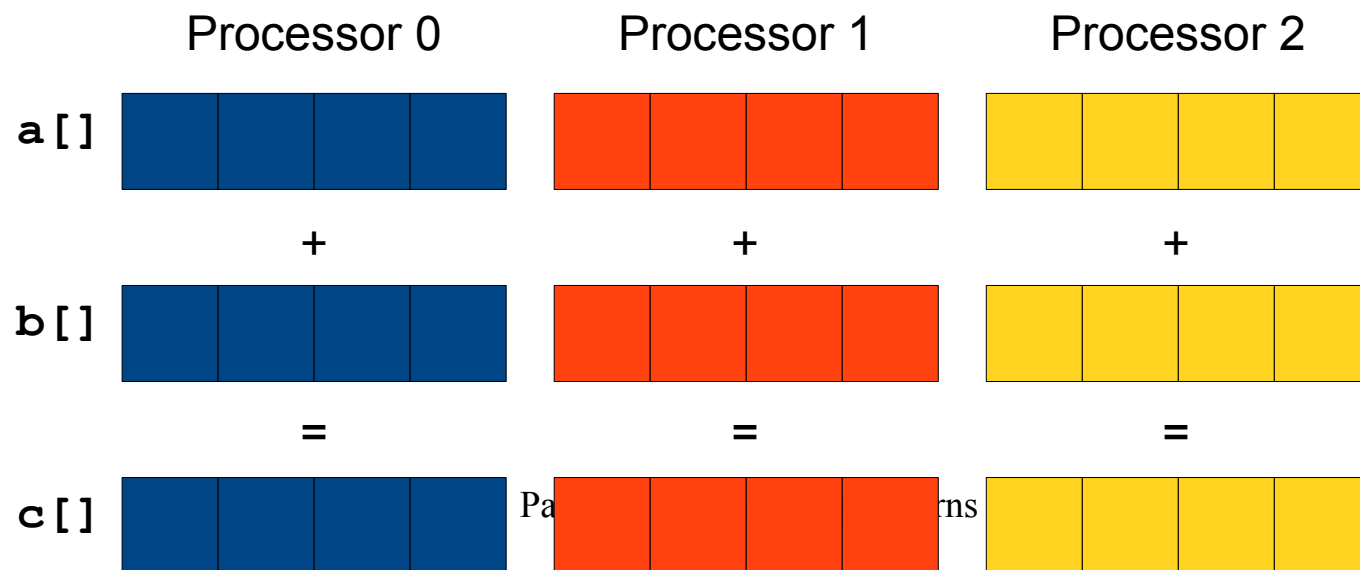


Parallel Programming Patterns

- Embarrassingly Parallel
- Partition
- Master-Worker
- Stencil
- Reduce
- Scan

Embarrassingly Parallel

- Applies when the computation can be decomposed in independent tasks that require little or no communication
- Examples:
 - Vector sum
 - Mandelbrot set
 - 3D rendering
 - Brute force password cracking
 - ...

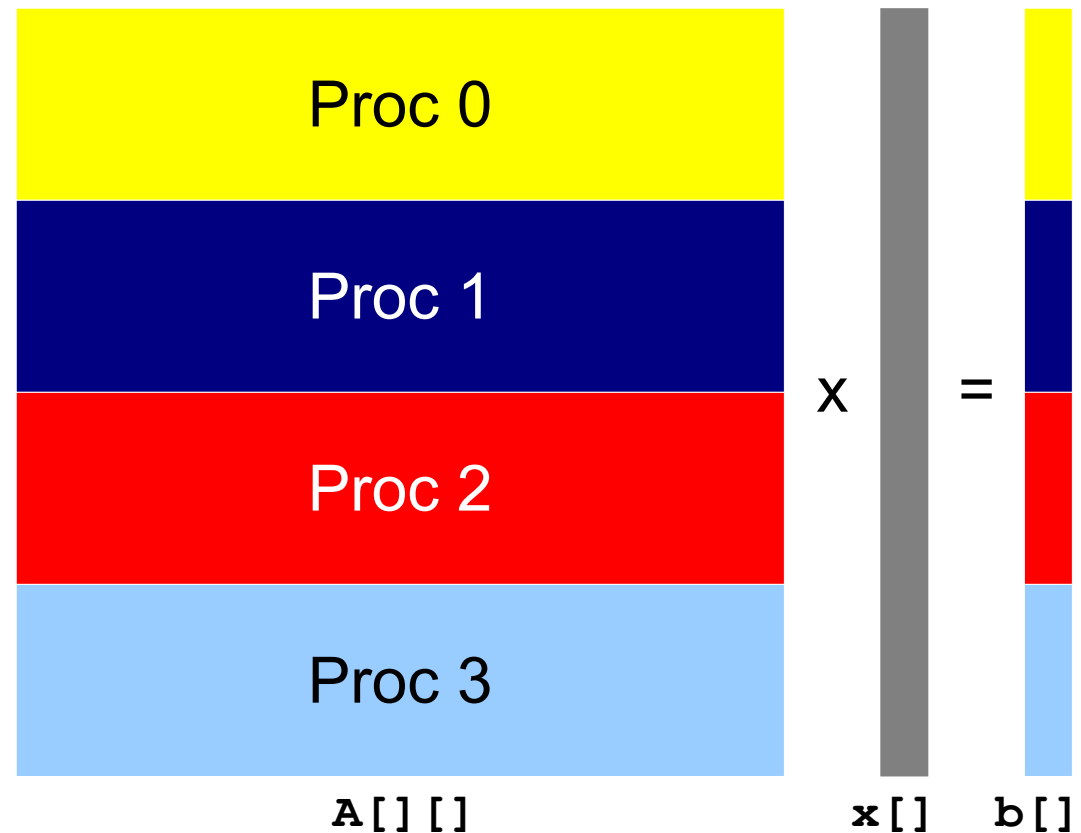


Partition

- The input data space (in short, *domain*) is split in disjoint regions called *partitions*
- Each processor operates on one partition
- This pattern is particularly useful when the application exhibits *locality of reference*
 - i.e., when processors can refer to their own partition only and need little or no communication with other processors

Example

- Matrix-vector product
 $Ax = b$
- Matrix $A[] []$ is partitioned into P horizontal blocks
- Each processor
 - operates on one block of $A[] []$ and on a full copy of $x[]$
 - computes a portion of the result $b[]$



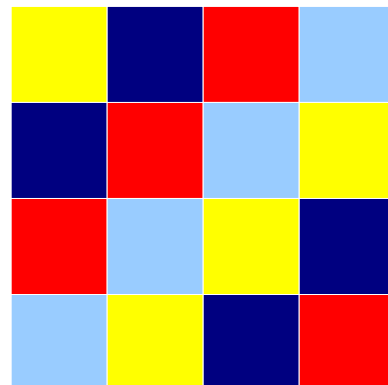
Regular vs Irregular partitioning

- Regular

- the domain is split into partitions of roughly the same size and shape

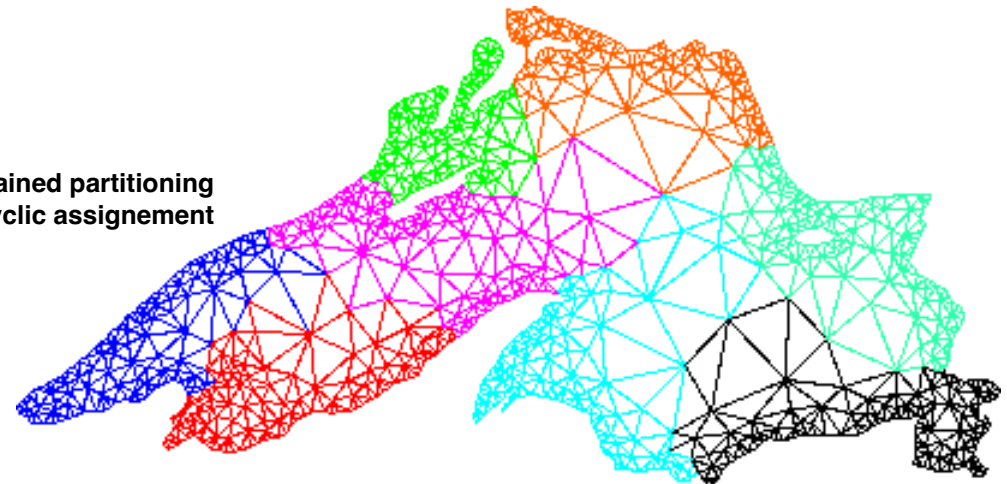


fine grained partitioning
with cyclic assignment



- Irregular

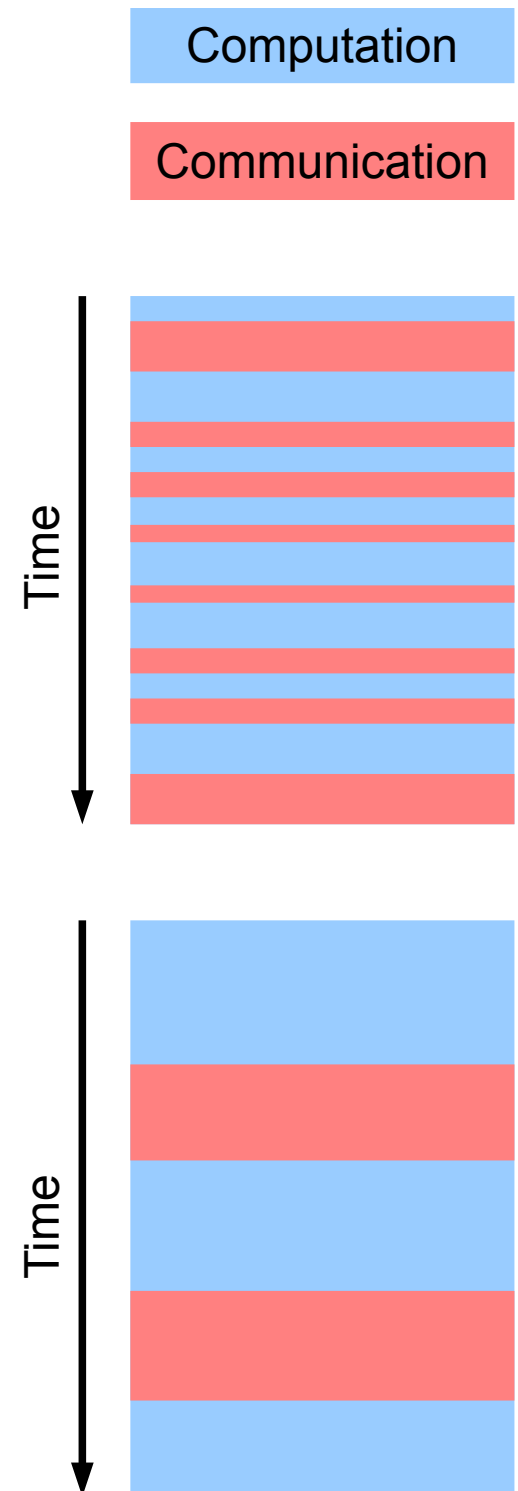
- partitions do not necessarily have the same size or shape



Here a lake is divided in triangles in order to then
assign an independent computation to each triangle

Fine grained vs Coarse grained partitioning

- *Fine-grained Partitioning*
 - Better load balancing, especially if combined with the master-worker pattern (see later)
 - If granularity is too fine, the computation / communication ratio might become too low (communication dominates on computation)
- *Coarse-grained Partitioning*
 - In general improves the computation / communication ratio
 - However, it might cause load imbalancing
- The "optimal" granularity is sometimes problem-dependent; in other cases the user must choose which granularity to use



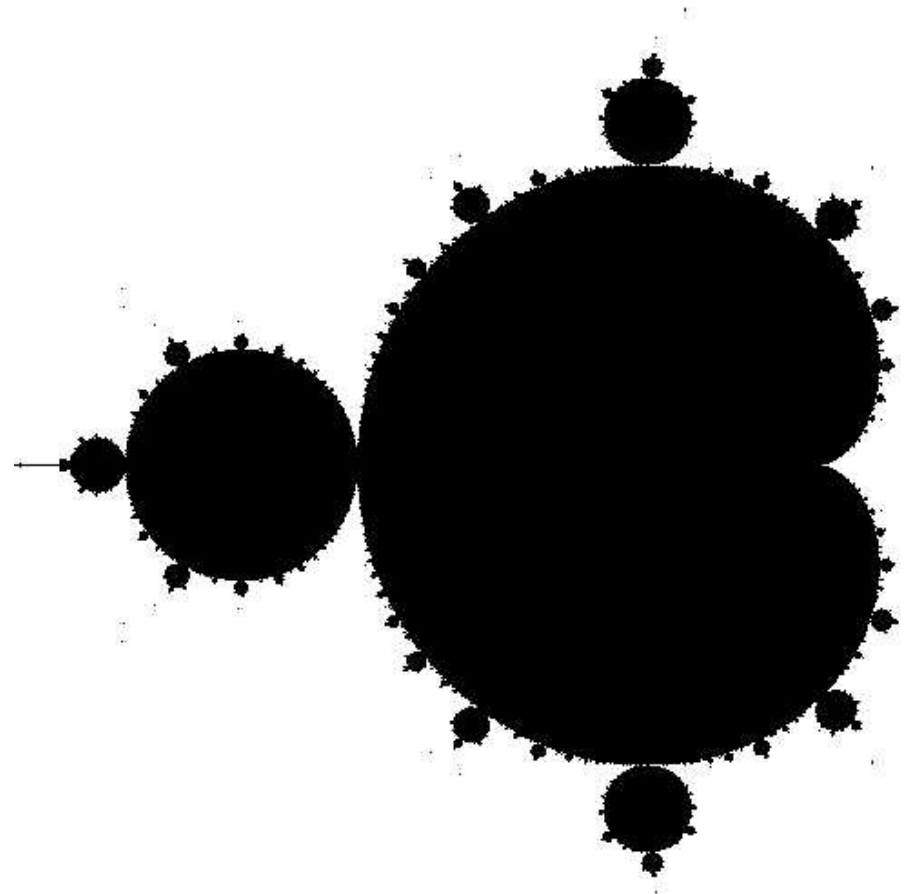
Example: Mandelbrot set

- The Mandelbrot set is the set of points c on the complex plane s.t. the sequence $z_n(c)$ defined as

$$z_n(c) = \begin{cases} 0 & \text{if } n=0 \\ z_{n-1}^2(c) + c & \text{otherwise} \end{cases}$$

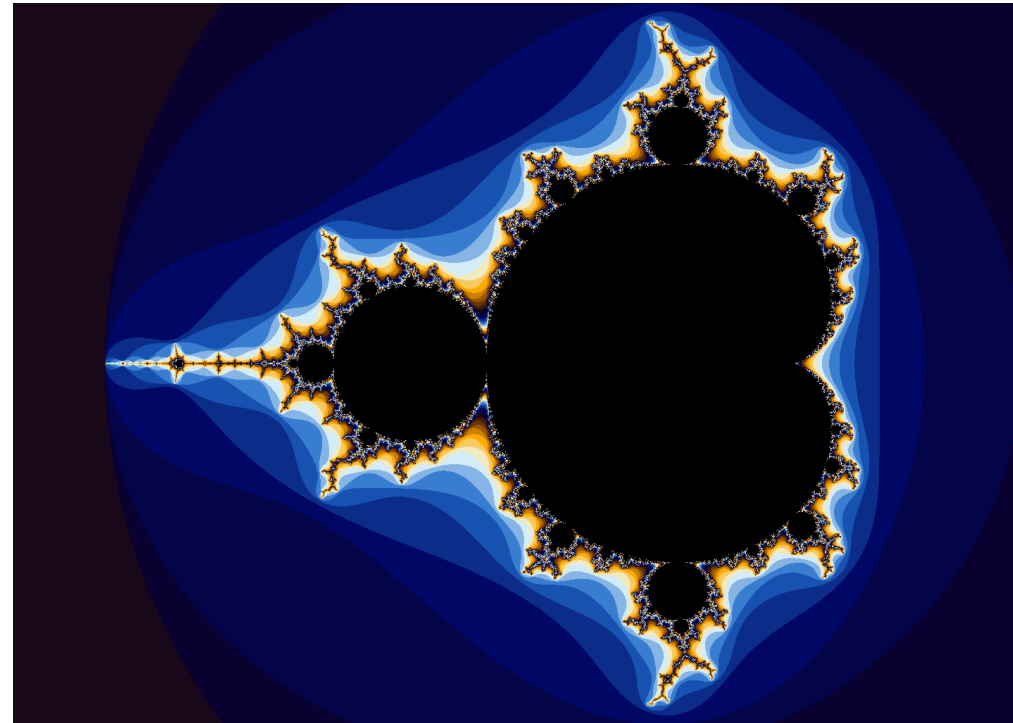
does not diverge when
 $n \rightarrow +\infty$

We will approximate it with `max_iter` computation



Mandelbrot set in color

- If the modulus of $z_n(c)$ does not exceed 2 after *maxit* iterations, the pixel is black (the point is assumed to be part of the Mandelbrot set)
- Otherwise, the color depends on the number of iterations required for the modulus of $z_n(c)$ to become > 2



The coloring is dependant of the number of iterations to get there and sometimes colors get reused by using module operations

The module of a complex number is the circle pointed in the origin of the Im/Re plane.

Pseudocode

Embarrassingly parallel structure: the color of each pixel can be computed independently from other pixels

```
maxit = 1000;
for each point (cx, cy) {
    x = y = 0;
    it = 0;
    while ( it < maxit AND  $x*x + y*y \leq 2*2$  ) {
        xnew = x*x - y*y + cx;
        ynew = 2*x*y + cy;
        x = xnew;
        y = ynew;
        it = it + 1;
    }
    plot(cx, cy, it);
}
```

Point (x,y) inside the module

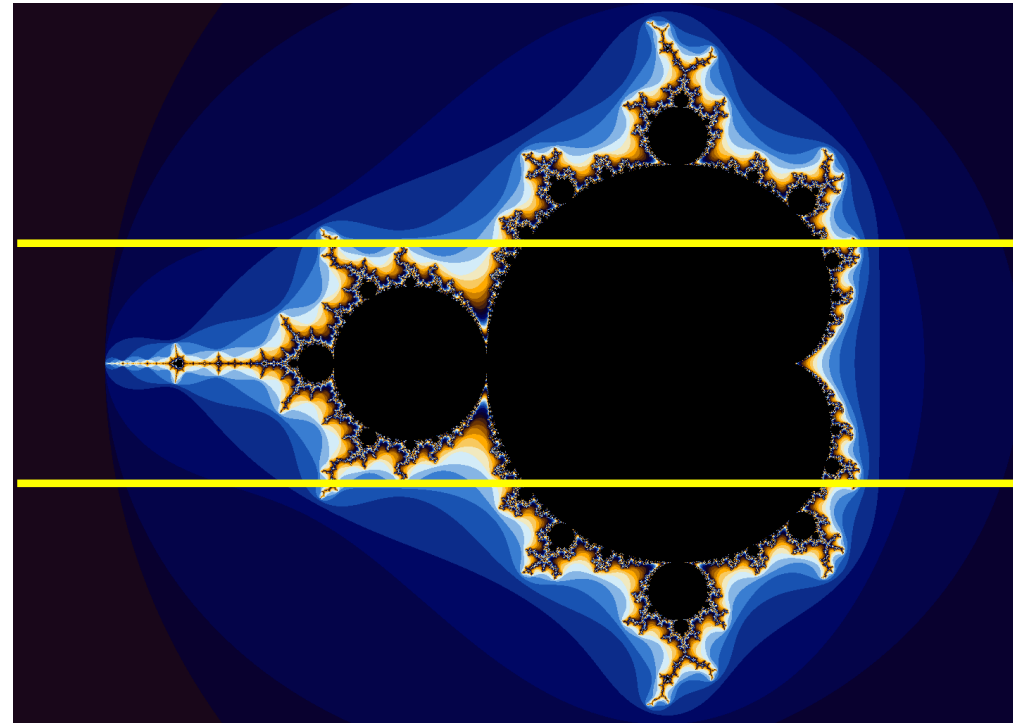
$$\begin{aligned} z(c)^2 + c &= \\ &= (x+iy)^2 + cx + icy \\ &= x^2 - y^2 + 2ixy + cx + icy \\ &= x^2 + y^2 + cx + i(2xy + cy) \end{aligned}$$

NB: cx,cy are x,y components of c

Source: http://en.wikipedia.org/wiki/Mandelbrot_set#For_programmers

Mandelbrot set

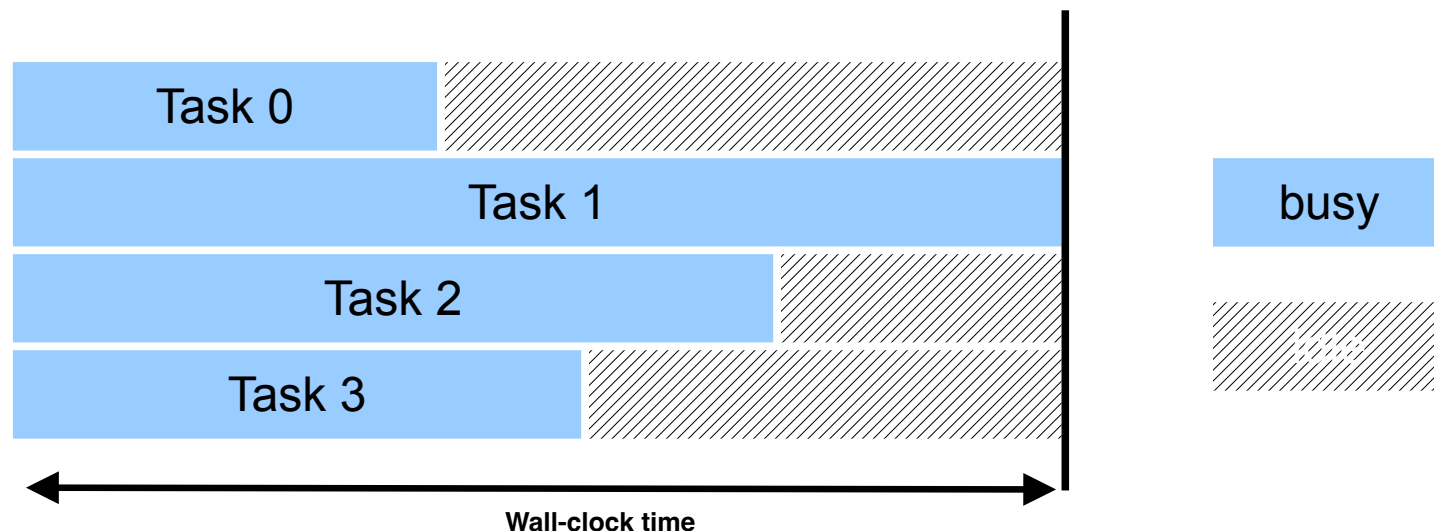
- A regular partitioning can result in uneven load distribution
 - Black pixels require *maxit* iterations
 - Other pixels require fewer iterations



We divide it into 3 partitions, we would be tempted to make them have equal size. Instead we can notice that if a point is not in the set it requires less time to compute it as it exits the loop. So the central part has more points in the set and so it will require more time than the others.
==> Consider load balancing partitioning

Load balancing

- Ideally, each processor should perform the same amount of work
 - If the tasks synchronize at the end of the computation, the execution time will be that of the **slower** task

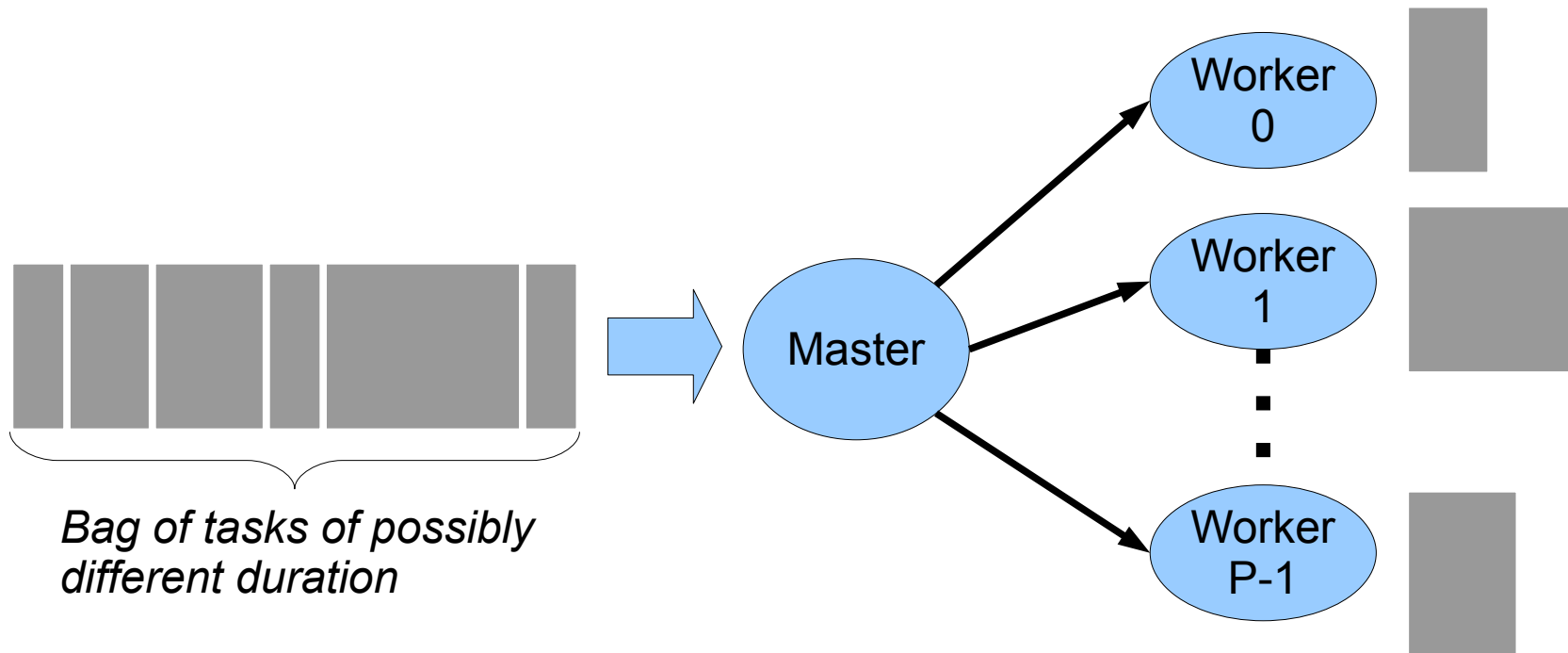


Load balancing howto

- The workload is balanced if each processor performs more or less the same amount of work
- How to achieve load balancing:
 - Use fine-grained partitioning
 - ...but beware of the possible communication overhead if the tasks need to communicate
 - Use dynamic task allocation (**master-worker** paradigm)
 - ...but beware that dynamic task allocation might incur in higher overhead with respect to static task allocation

Master-worker paradigm (*process farm, work pool*)

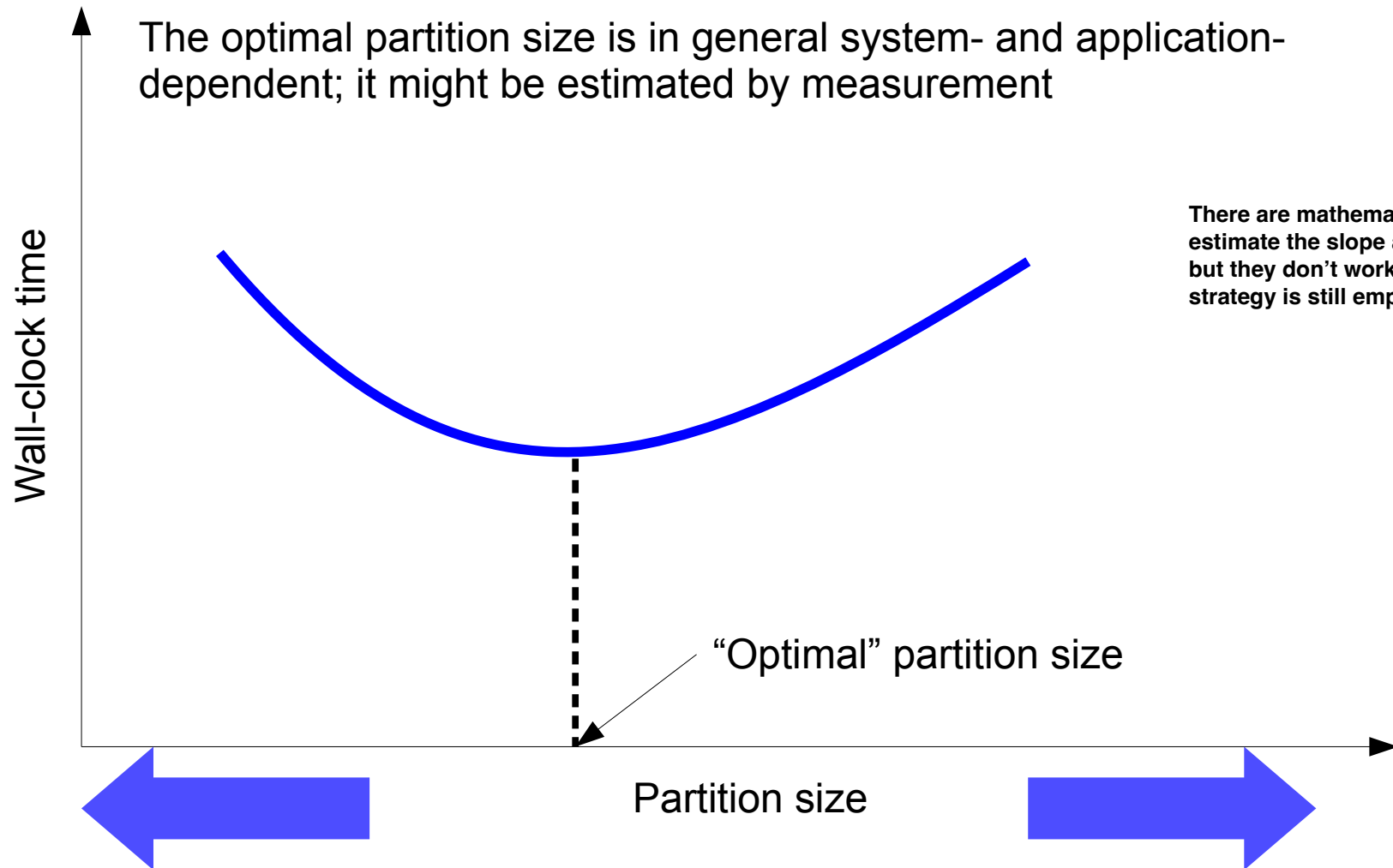
- Apply a fine-grained partitioning
 - *number of tasks* $\gg$ *number of execution units*
- The master assigns a task to the first available worker



Choosing the partition size

The optimal partition size is in general system- and application-dependent; it might be estimated by measurement

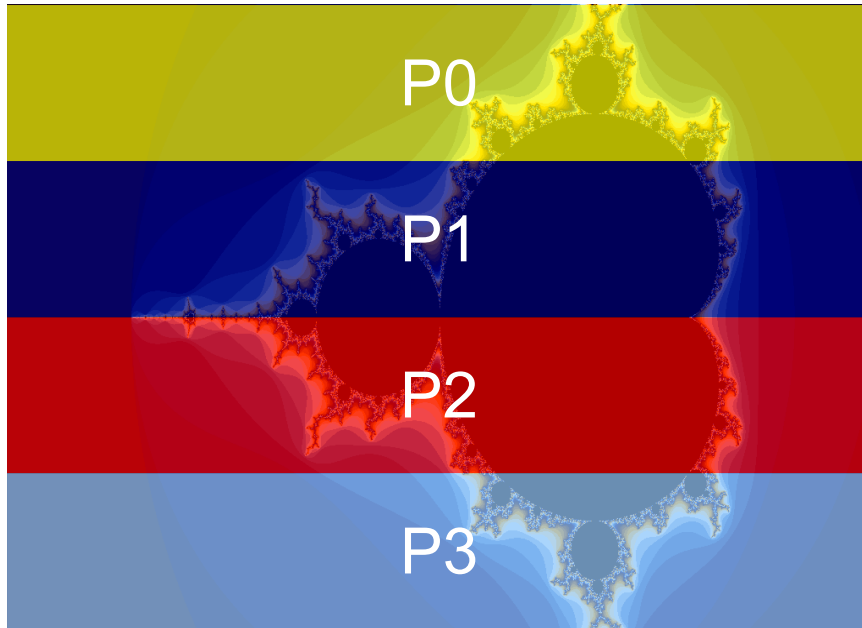
There are mathematical models to estimate the slope and so the optima but they don't work so well so the best strategy is still empirical



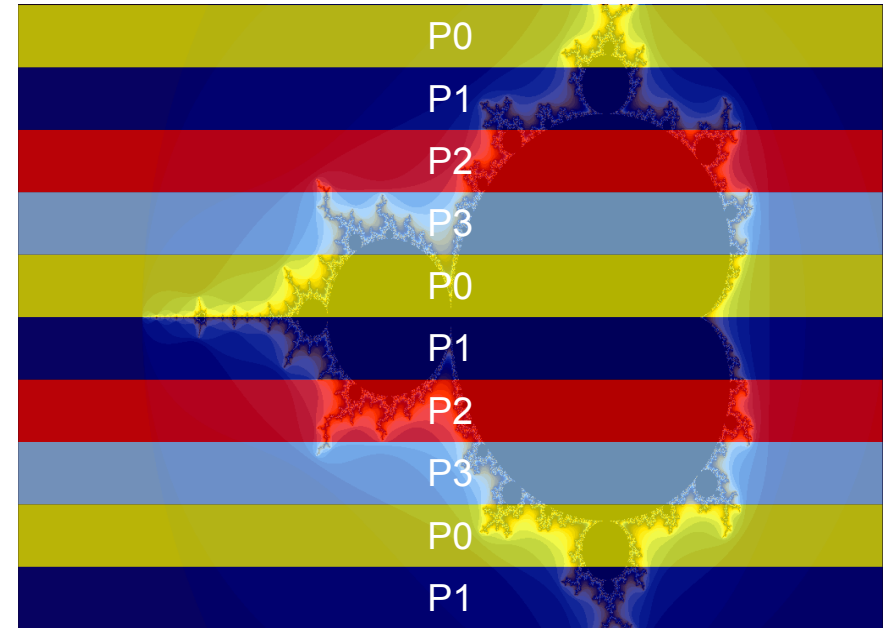
Too small = higher scheduling overhead

Too large = unbalanced workload

coarse-grained decomposition
static task assignment

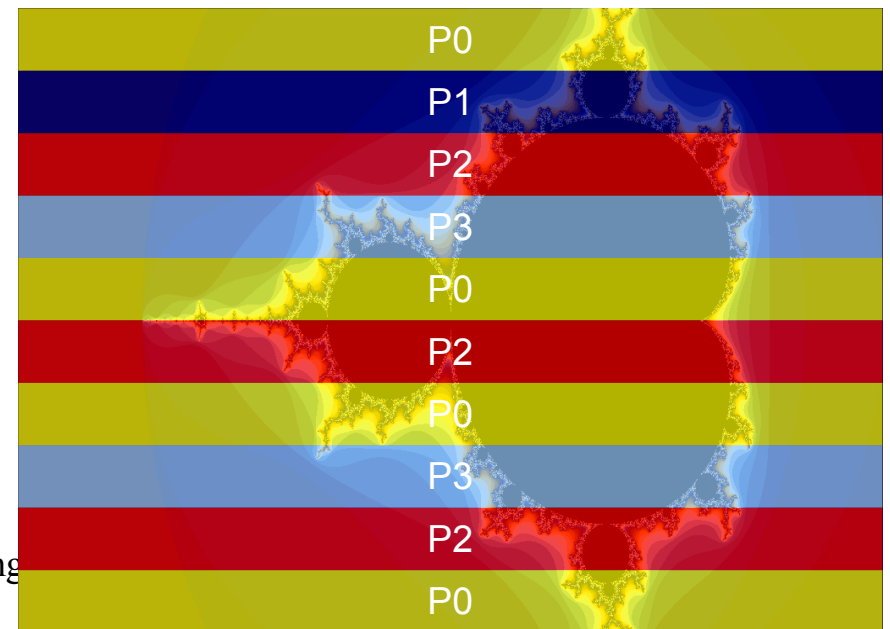


block size = 64
static task assignment



block size = 64
dynamic (master-worker) task assignment
(example)

The colors are random as we can't
predict which process would finish
before and start to take the next
portion



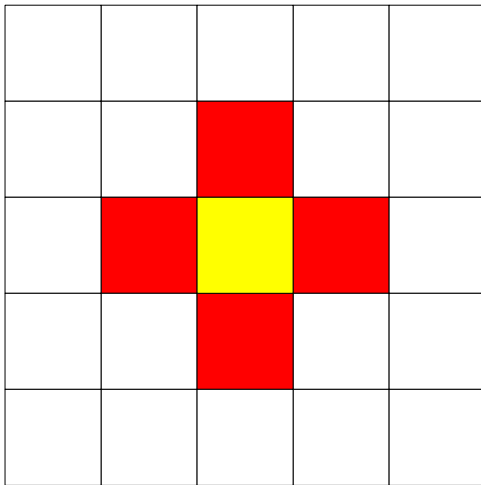
Stencils

- Stencil computations involve a grid whose values are updated according to a fixed pattern called *stencil*
 - Example: the Gaussian smoothing of an image updates the color of each pixel with the weighted average of the previous colors of the 5×5 neighborhood

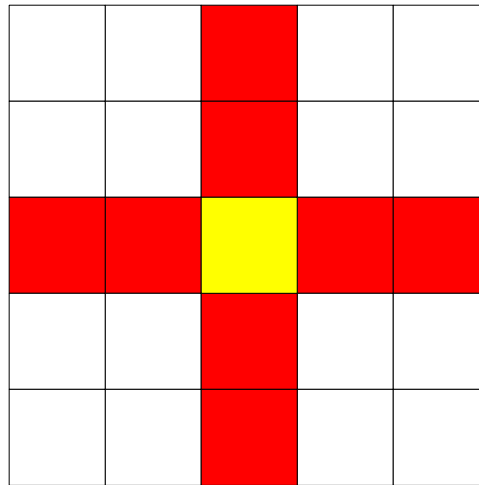
	1	4	5	4	1	
	4	16	28	16	4	
	7	28	41	28	7	
	4	16	28	16	4	
	1	4	7	4	1	

2D Stencils

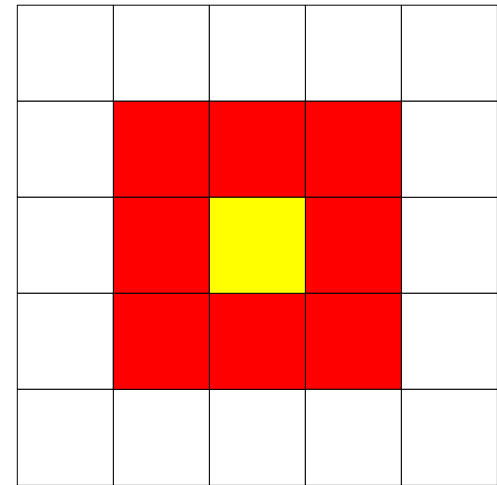
5-point 2-axis 2D stencil
(von Neumann neighborhood)



9-point 2-axis 2D stencil



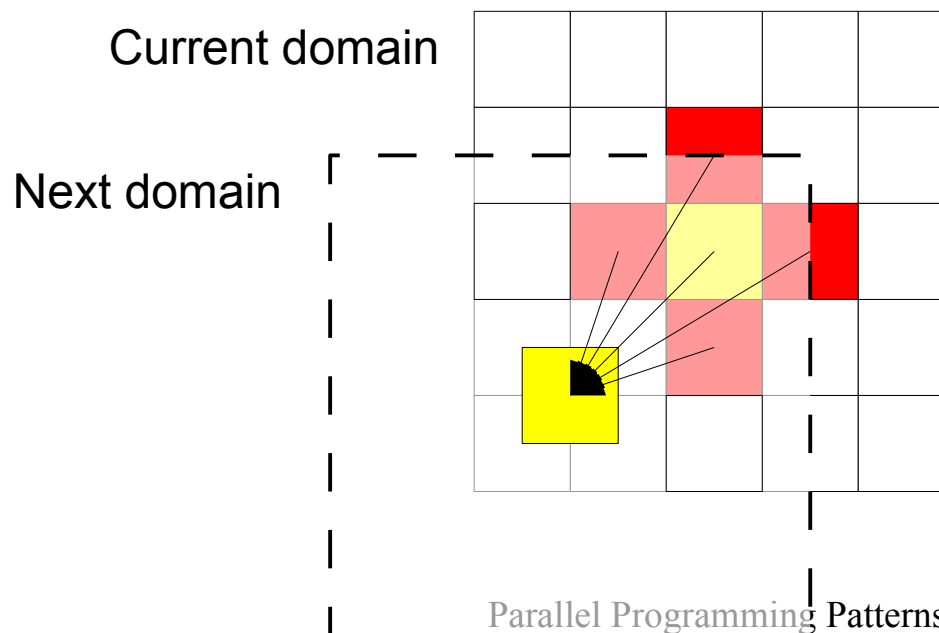
9-point 1-plane 2D stencil
(Moore neighborhood)



Stencils

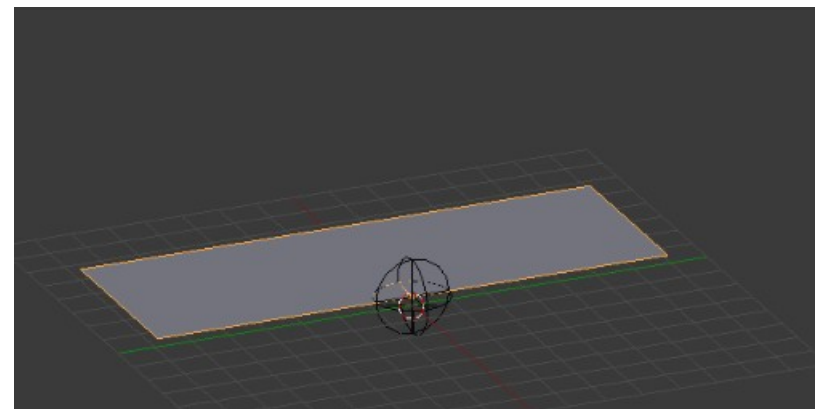
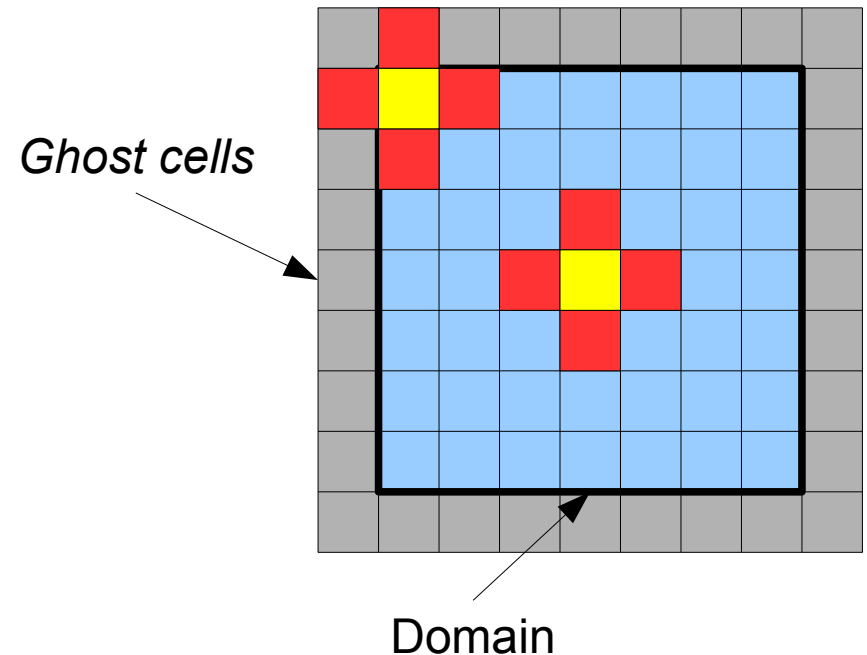
2 buffers

- Stencil computations usually employ **two** domains to keep the **current** and **next** values
 - Values are **read** from the **current** domain
 - New values are **written** to the **next** domain
 - **current** and **next** are exchanged at the end of each step



Ghost Cells

- How do we handle cells on the border of the domain?
 - We might assume that cells outside the border have some fixed, application-dependent value, or
 - We may assume **periodic boundary conditions**, where sides are “glued” together to form a torus
- In either case, we extend the domain with **ghost cells**, so that cells on the border do not require any special treatment



Parallelizing stencil computations

- Computing the next grid from the current one has embarrassingly parallel structure

```
Initialize current grid  
while (!terminated) {  
    Fill ghost cells  
    Compute next grid  
    Exchange current and next grids  
}
```

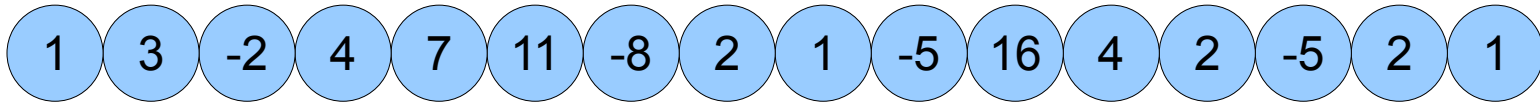
**Embarassingly
Parallel**



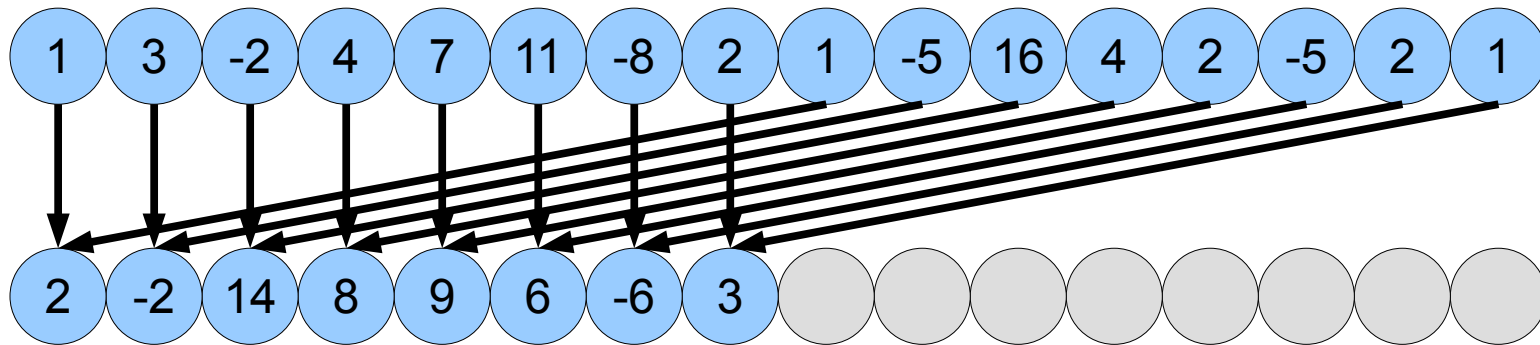
Reduce

- A **reduction** is the application of an **associative binary operator** (e.g., sum, product, min, max...) to the elements of an array $[x_0, x_1, \dots, x_{n-1}]$
 - $\text{sum-reduce}([x_0, x_1, \dots, x_{n-1}]) = x_0 + x_1 + \dots + x_{n-1}$
 - $\text{min-reduce}([x_0, x_1, \dots, x_{n-1}]) = \min \{ x_0, x_1, \dots, x_{n-1} \}$
 - ...
- A reduction can be realized in $O(\log_2 n)$ parallel steps

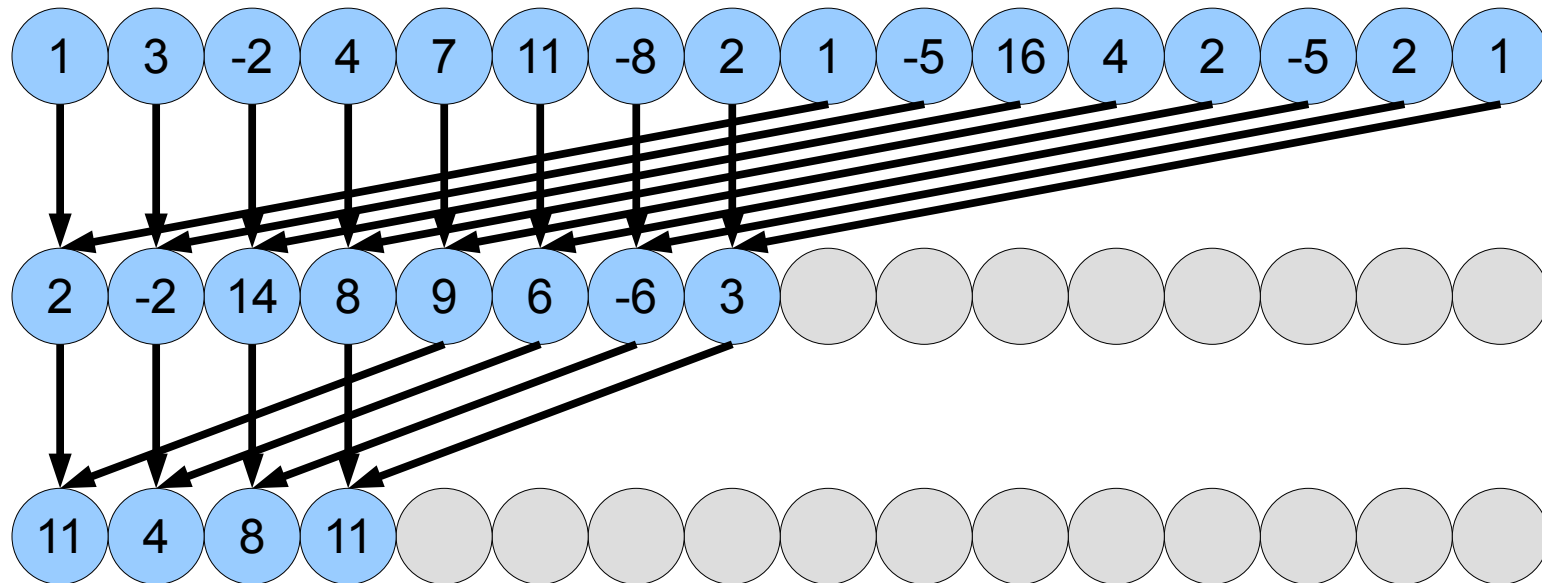
Example: sum-reduce



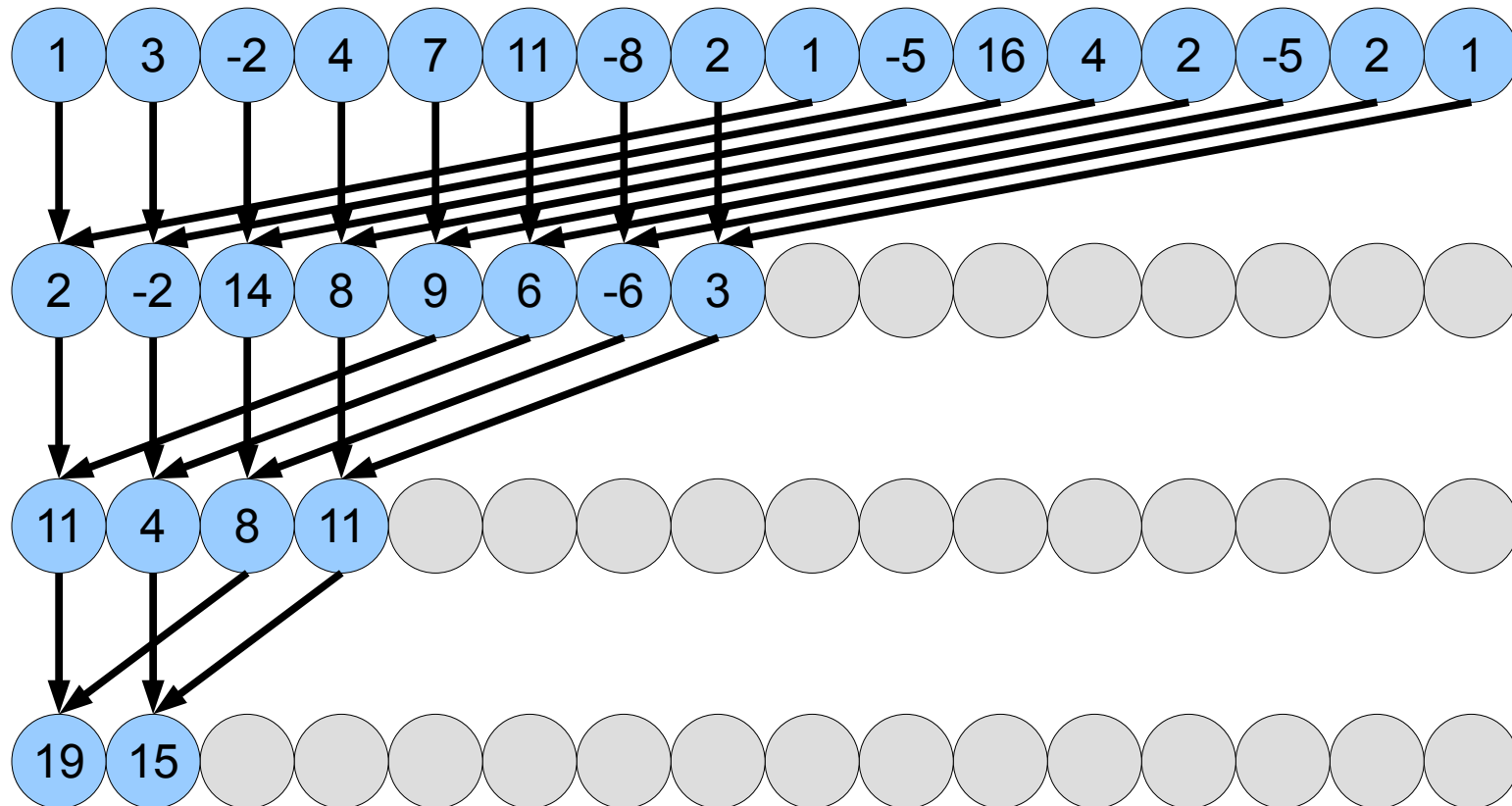
Example: sum-reduce



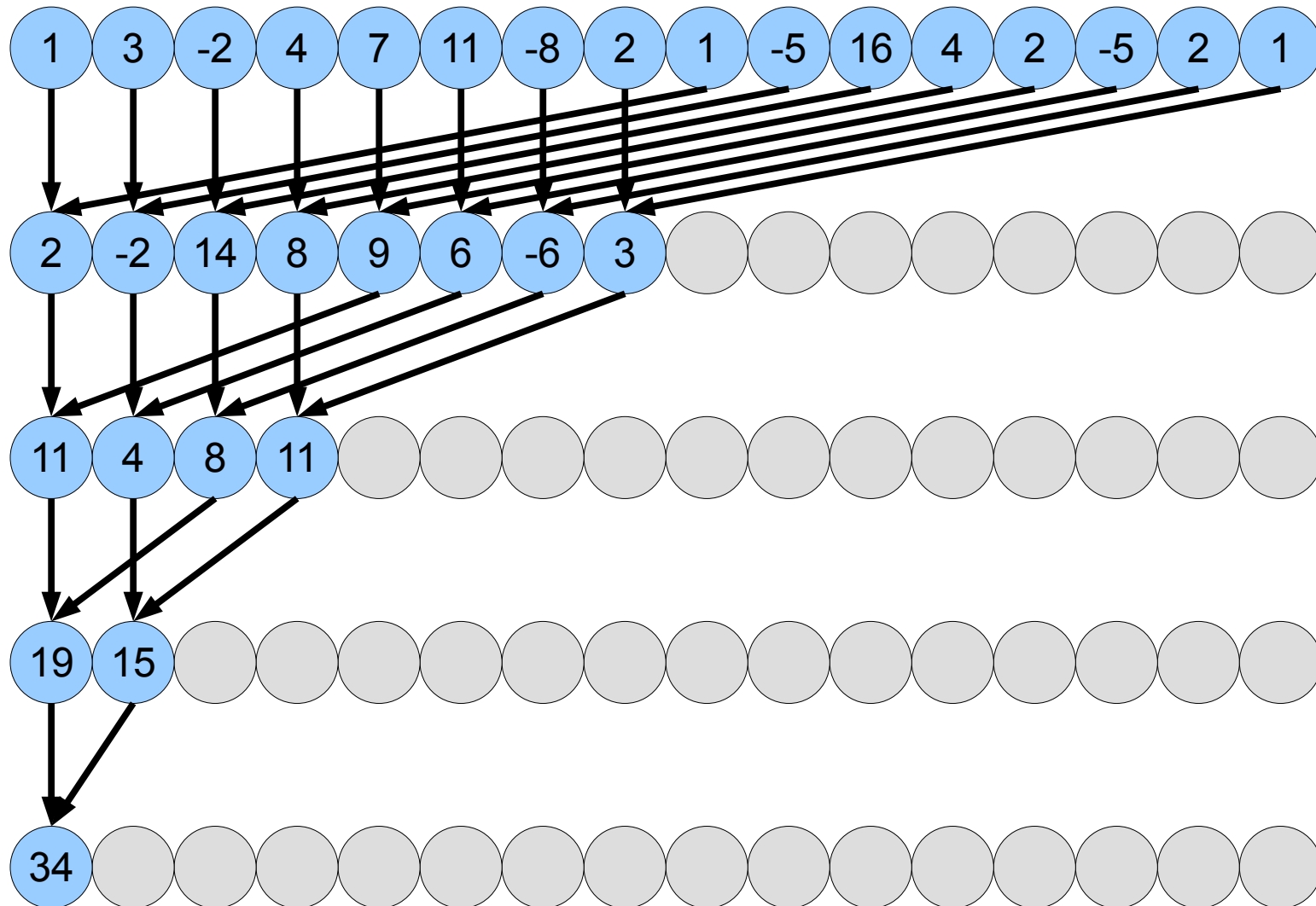
Example: sum-reduce



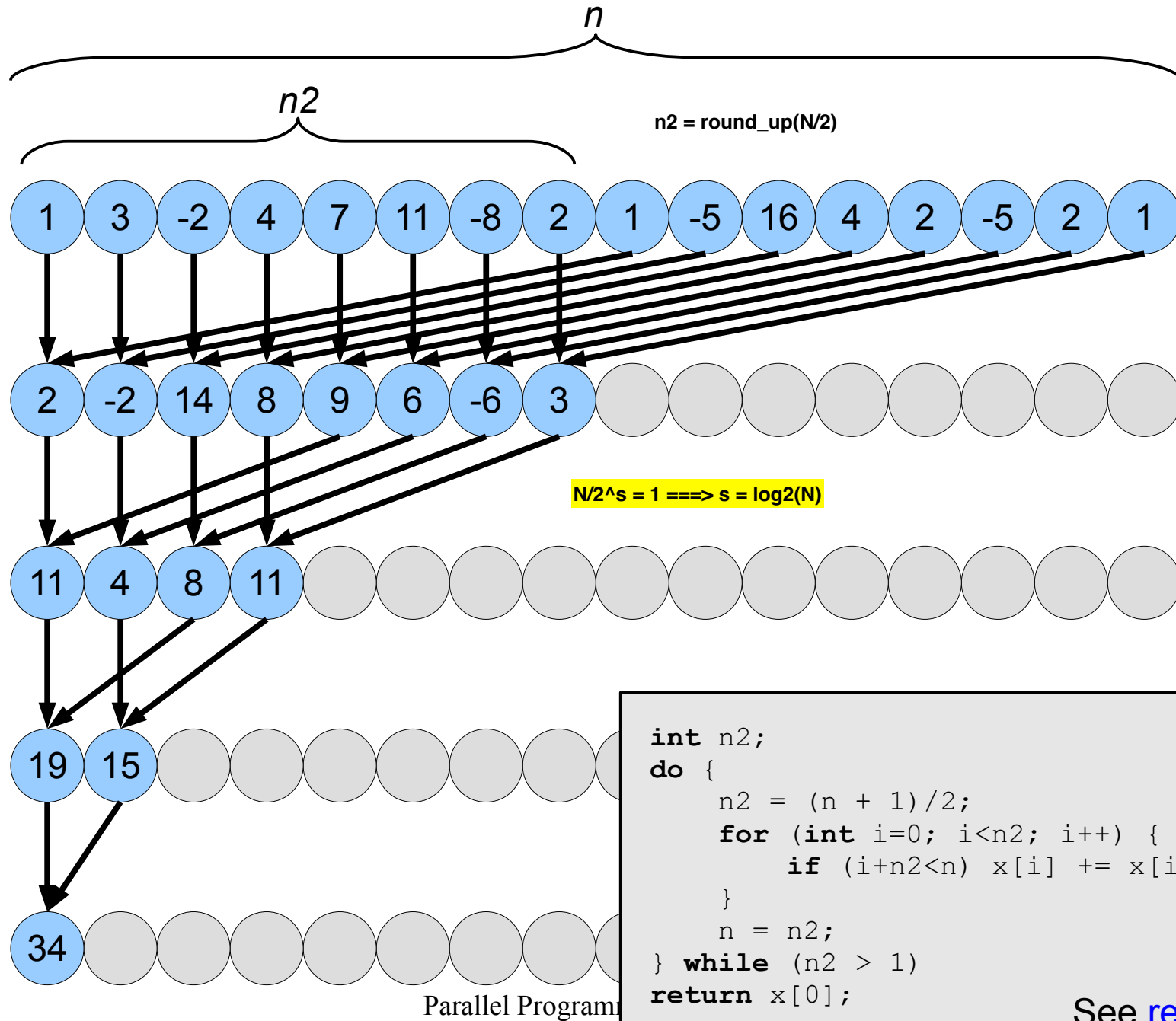
Example: sum-reduce



Example: sum-reduce



Example: sum-reduce



```
int n2;
do {
    n2 = (n + 1)/2;
    for (int i=0; i<n2; i++) {
        if (i+n2<n) x[i] += x[i+n2];
    }
    n = n2;
} while (n2 > 1)
return x[0];
```

See [reduction.c](#)

Scan (Prefix Sum)

- A **scan** computes all prefixes of an array $[x_0, x_1, \dots, x_{n-1}]$ using a given associative binary operator op (e.g., sum, product, min, max...)

$$[y_0, y_1, \dots, y_{n-1}] = \text{inclusive-scan}(op, [x_0, x_1, \dots, x_{n-1}])$$

where

$$y_0 = x_0$$

$$y_1 = x_0 \ op \ x_1$$

$$y_2 = x_0 \ op \ x_1 \ op \ x_2$$

...

$$y_{n-1} = x_0 \ op \ x_1 \ op \ \dots \ op \ x_{n-1}$$

Scan (Prefix Sum)

- A **scan** computes all prefixes of an array $[x_0, x_1, \dots, x_{n-1}]$ using a given associative binary operator op (e.g., sum, product, min, max...)

$$[y_0, y_1, \dots, y_{n-1}] = \text{exclusive-scan}(op, [x_0, x_1, \dots, x_{n-1}])$$

where

$$y_0 = 0$$

$$y_1 = x_0$$

exlude 2 in operators

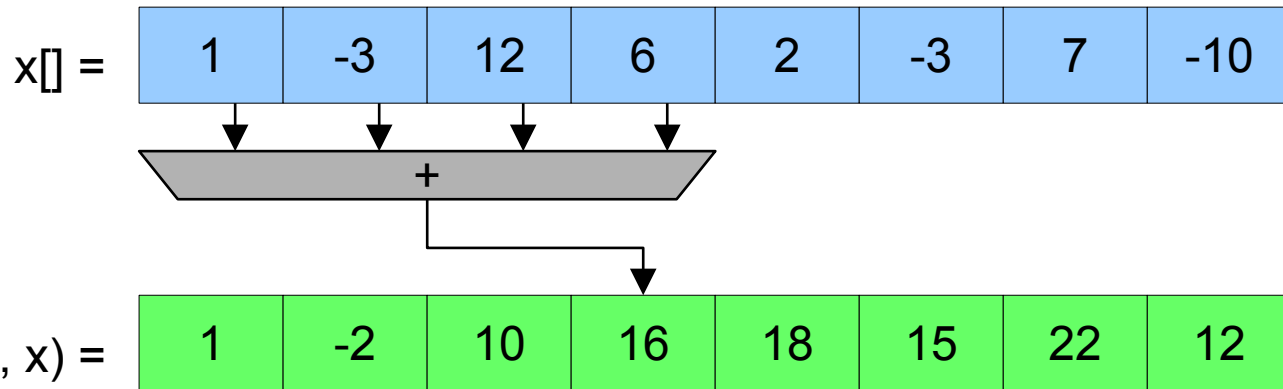
$$y_2 = x_0 \ op \ x_1$$

...

$$y_{n-1} = x_0 \ op \ x_1 \ op \ \dots \ op \ x_{n-2}$$

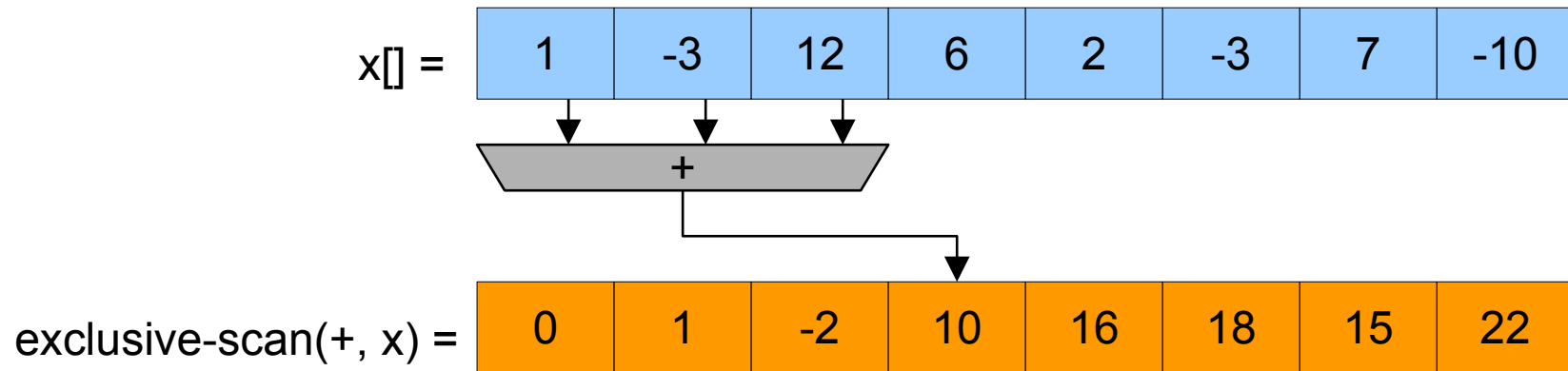
this is the neutral element of the binary operator (zero for sum, 1 for product, ...)

Example



Of course you can reuse the previous value for the next, making it bounded to the input length linearly. So we step from $O(n^2)$ to $O(n)$.

Example



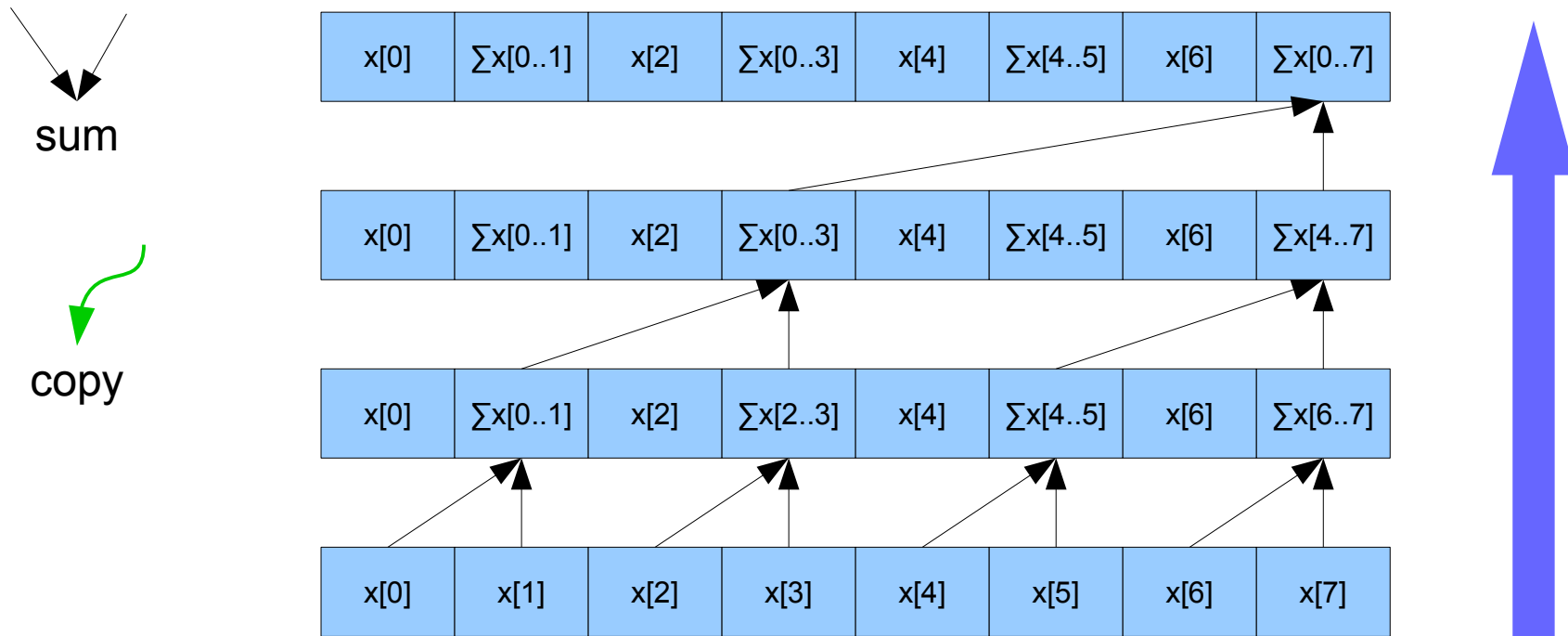
Serial implementation

```
void inclusive_scan(int *x, int *s, int n) // n must be > 0
{
    int i;
    s[0] = x[0];
    for (i=1; i<n; i++) {
        s[i] = s[i-1] + x[i];
    }
}
```

```
void exclusive_scan(int *x, int *s, int n) // n must be > 0
{
    int i;
    s[0] = 0;
    for (i=1; i<n; i++) {
        s[i] = s[i-1] + x[i-1];
    }
}
```

As we can notice the loops must be executed sequentially as the result for a loop is required from the next one.

Exclusive scan: Up-sweep

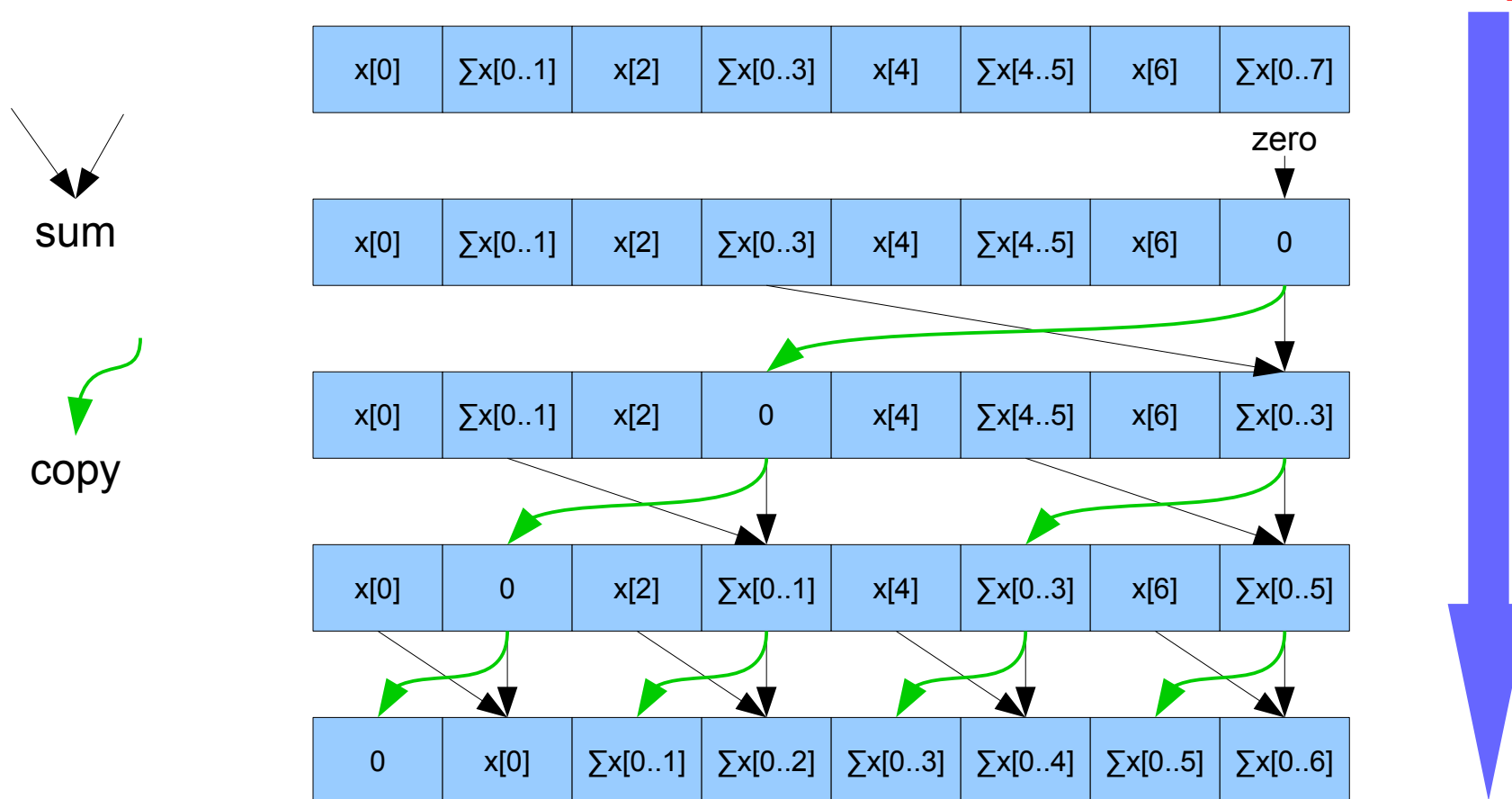


```
for ( d=1; d<n/2; d *= 2 ) {  
    for ( k=0; k<n; k+=2*d ) {  
        x[k+2*d-1] = x[k+d-1] + x[k+2*d-1];  
    }  
}
```

$O(\log_2 n)$ parallel steps

*Iterations of
this loop can
be executed
in parallel*

Exclusive scan: Down-sweep



```

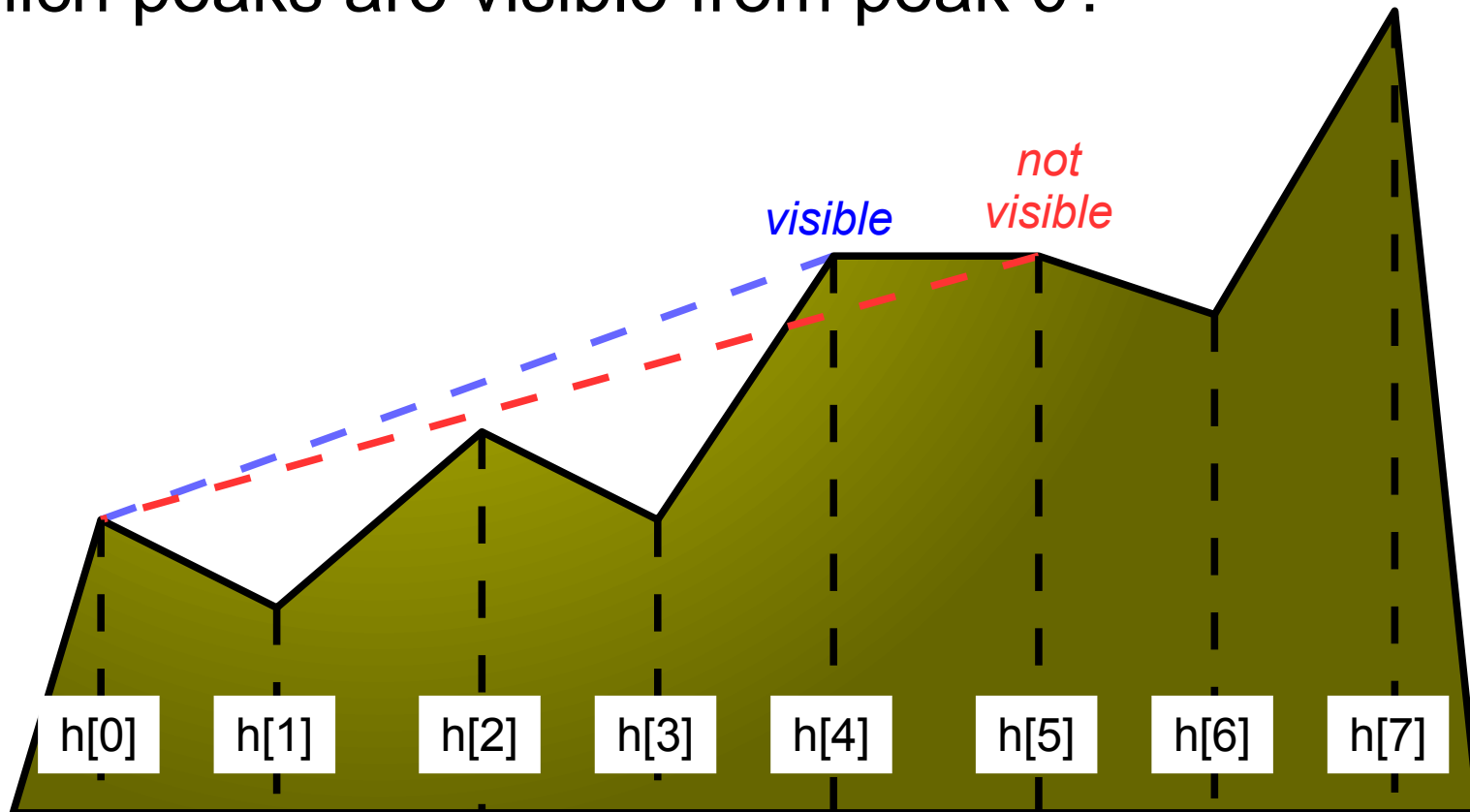
x[n-1] = 0;
for ( ; d > 0; d >>= 1 ) {
    for (k=0; k<n; k += 2*d ) {
        float t = x[k+d-1];
        x[k+d-1] = x[k+2*d-1];
        x[k+2*d-1] = t + x[k+2*d-1];
    }
}
    
```

Iterations of this loop can be executed in parallel

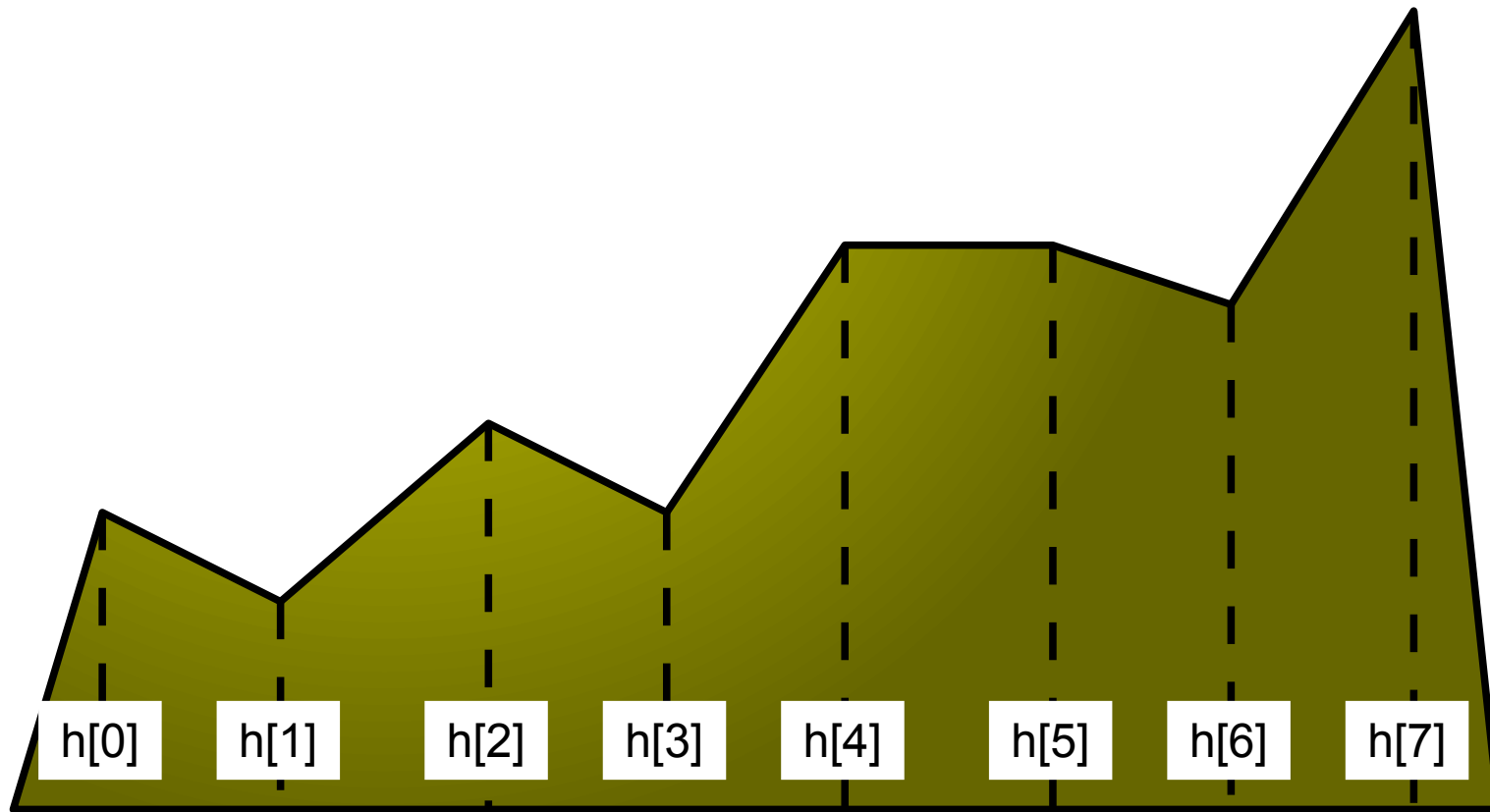
$O(\log_2 n)$ parallel steps

Example: Line of Sight

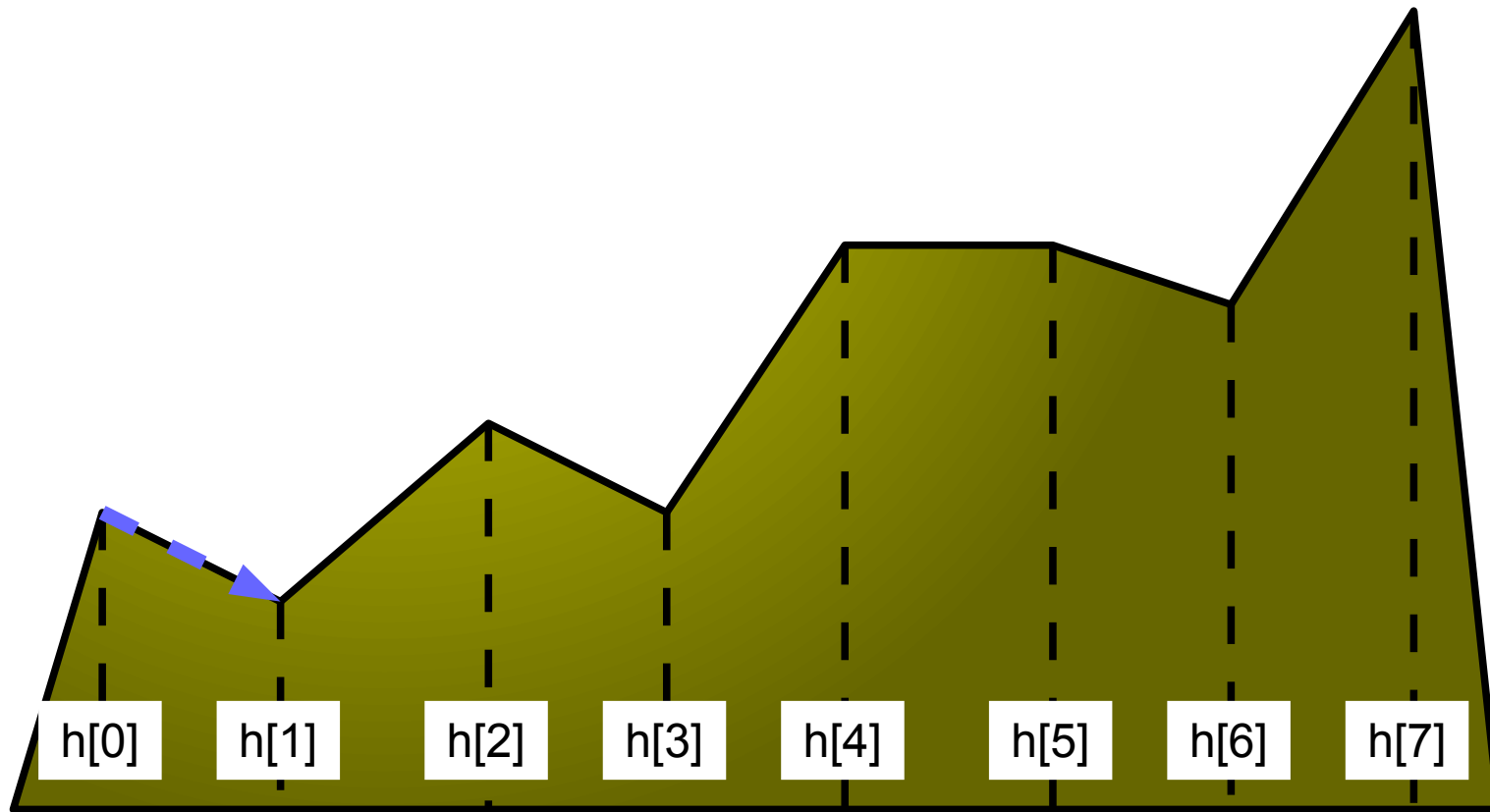
- n peaks of heights $h[0], \dots, h[n - 1]$; the distance between consecutive peaks is one
- Which peaks are visible from peak 0?



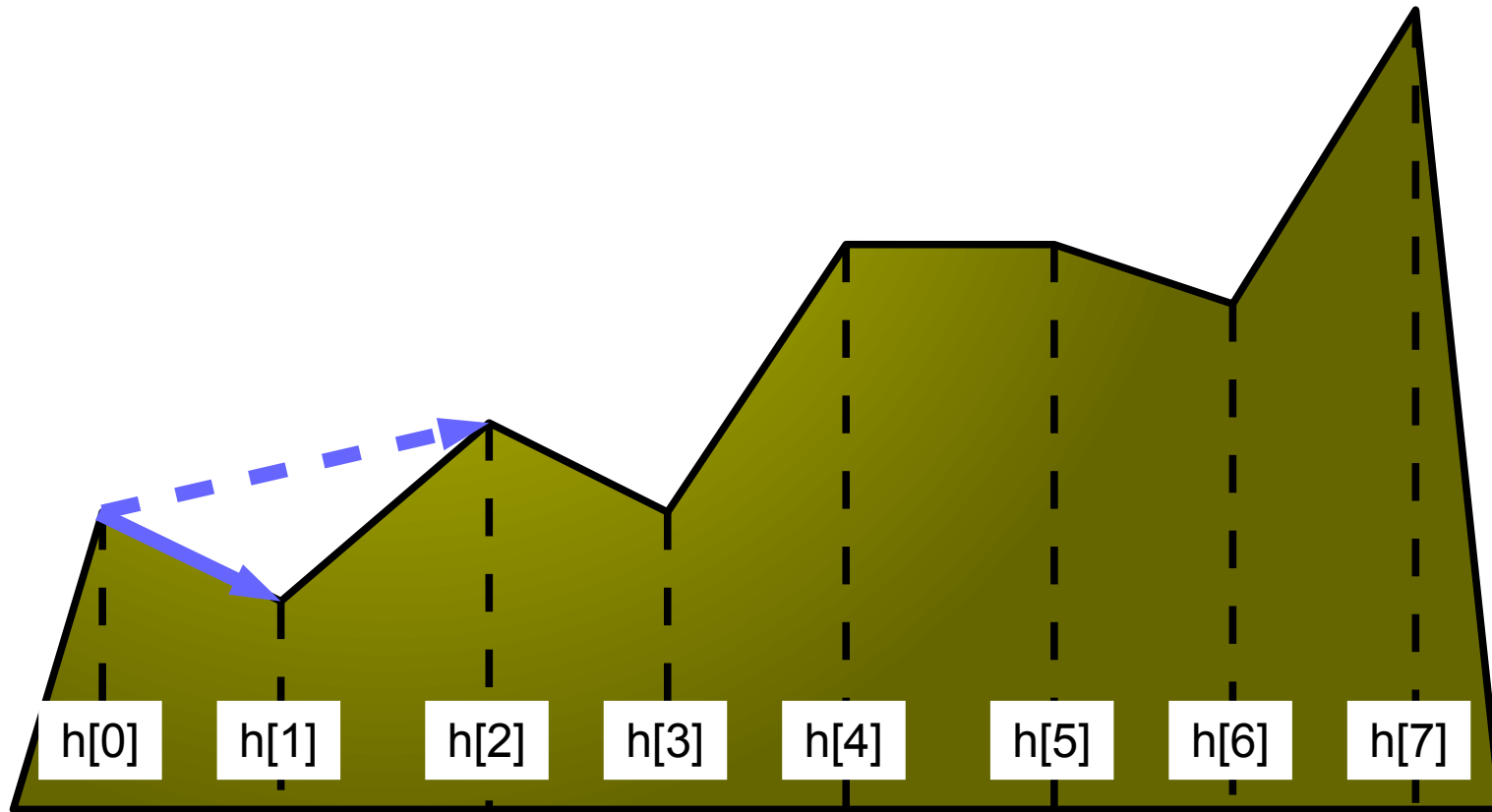
Line of sight



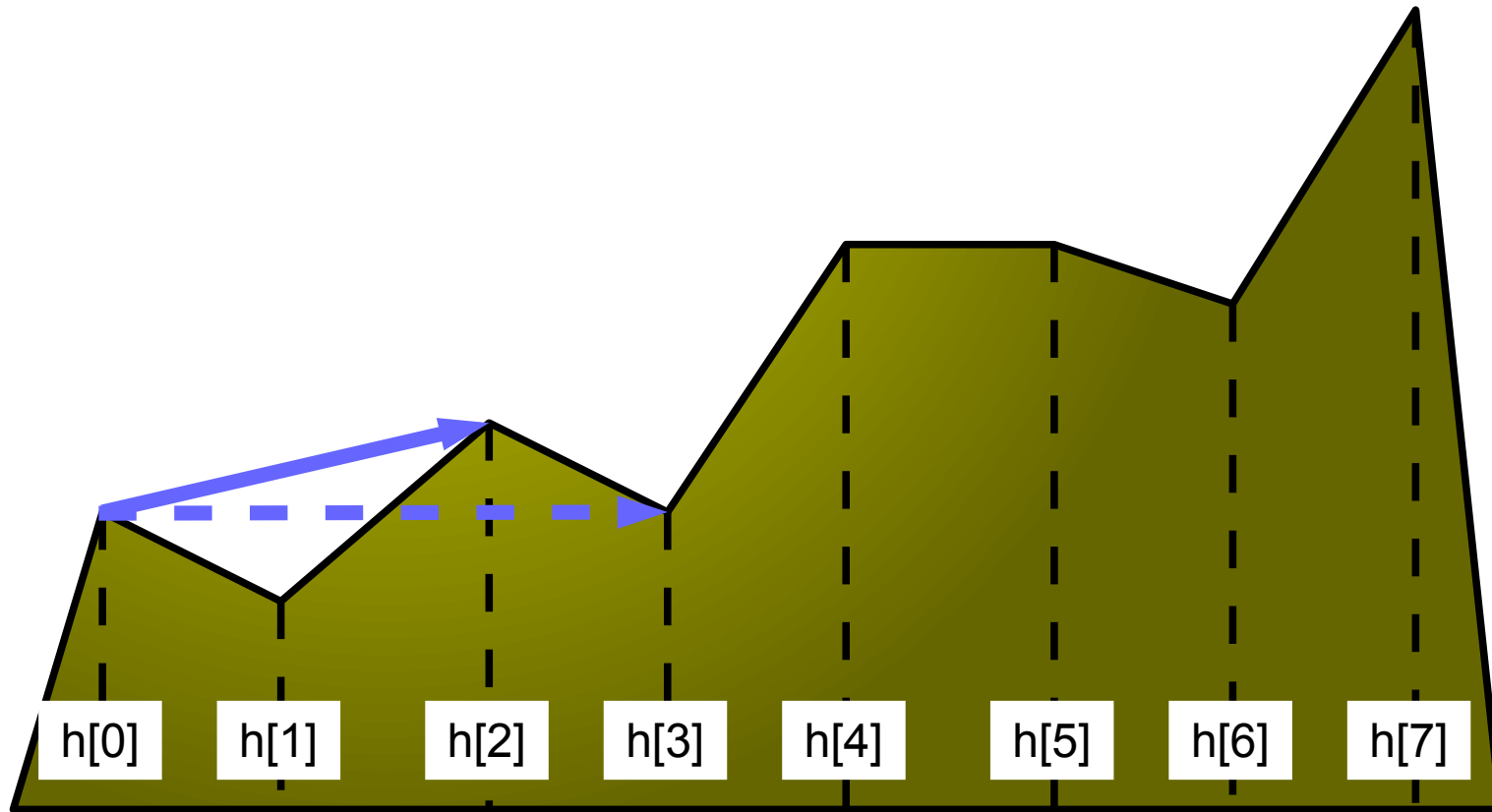
Line of sight



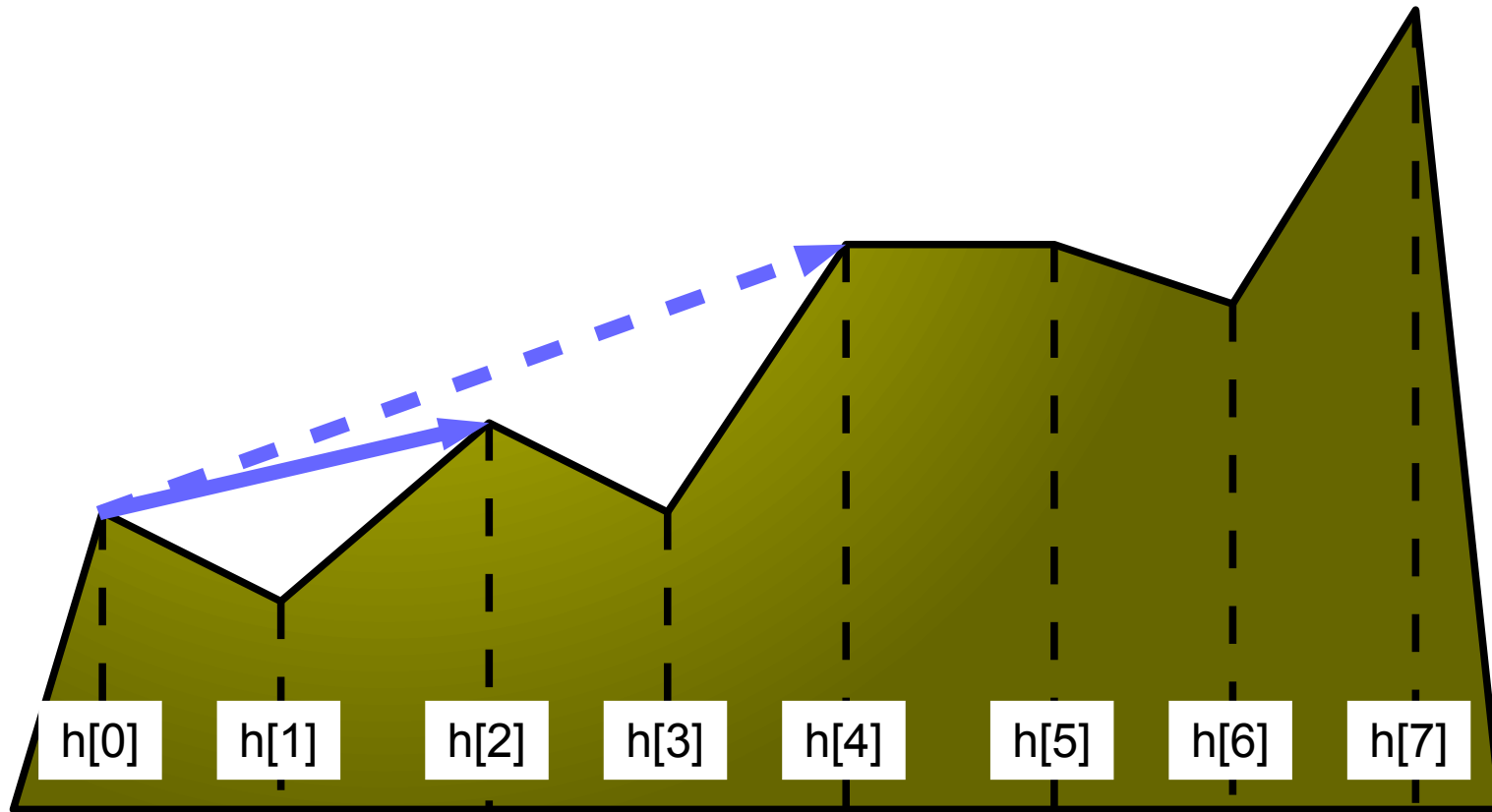
Line of sight



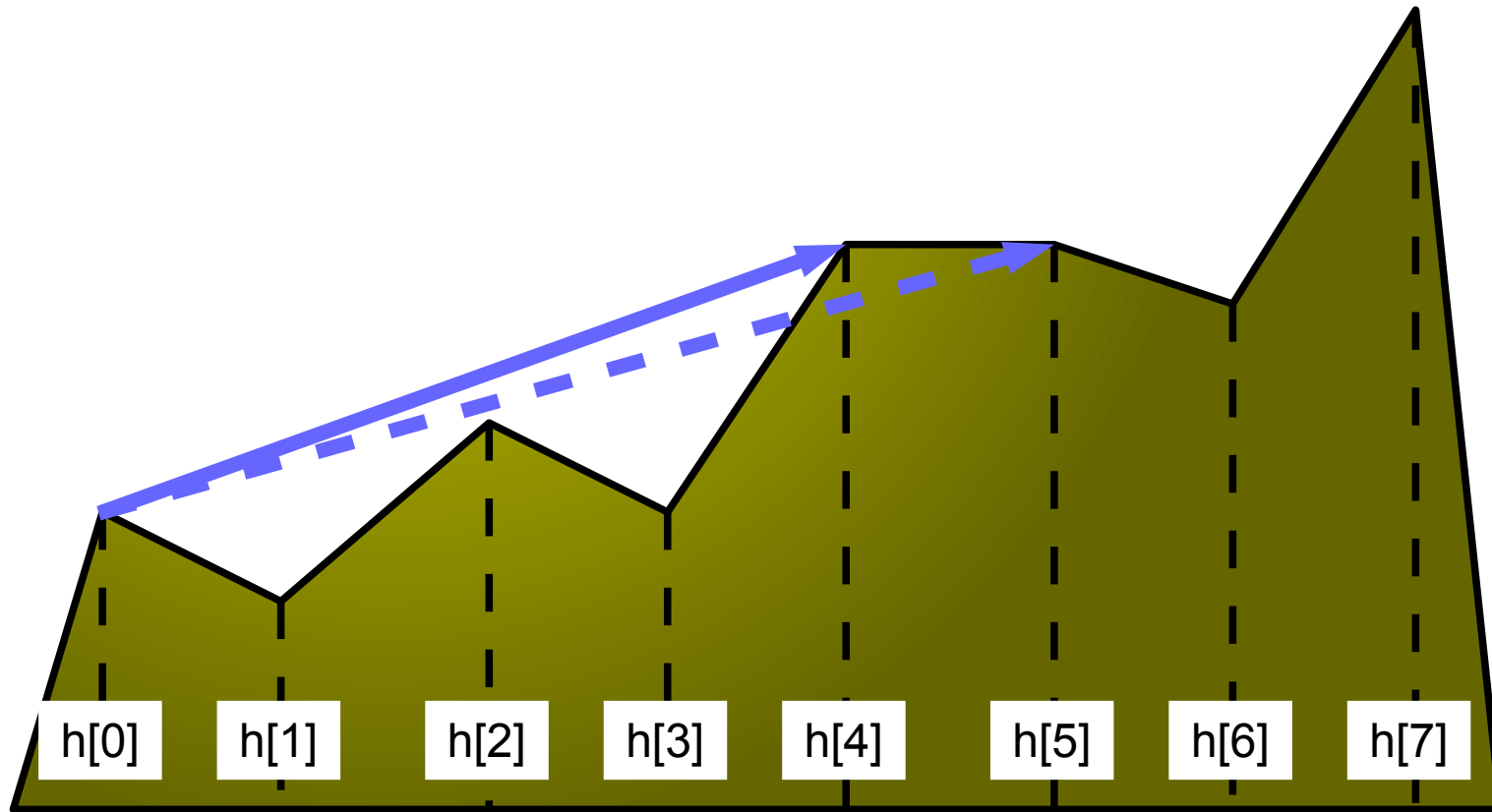
Line of sight



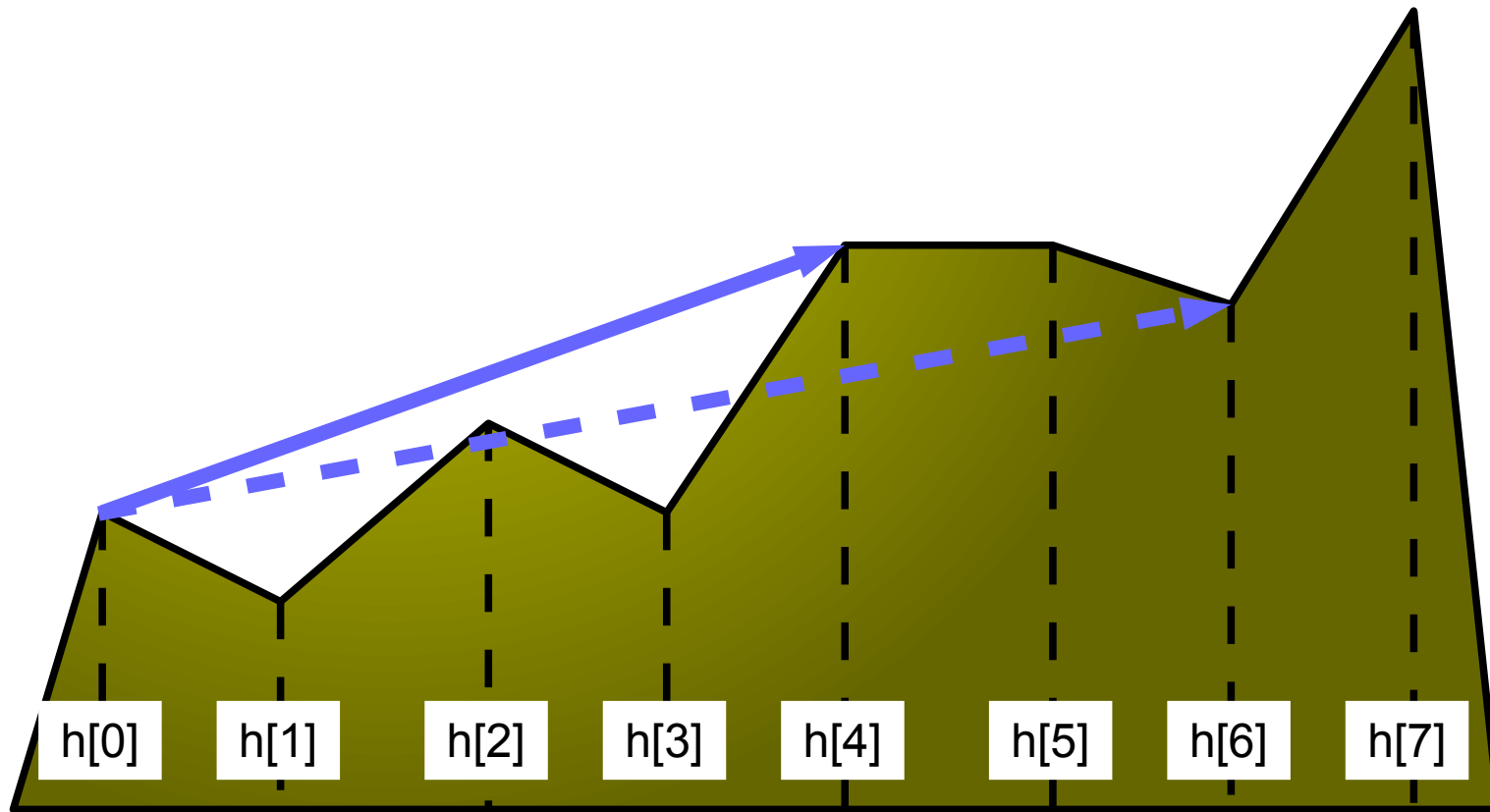
Line of sight



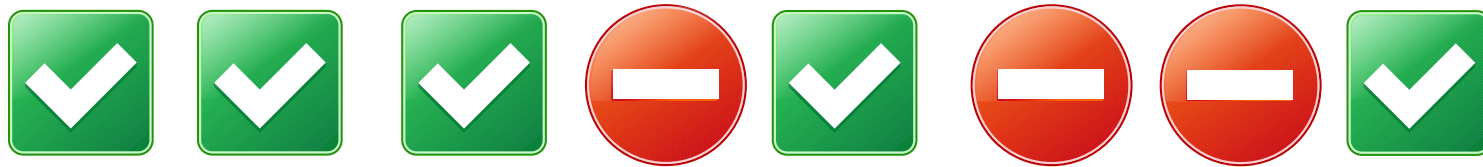
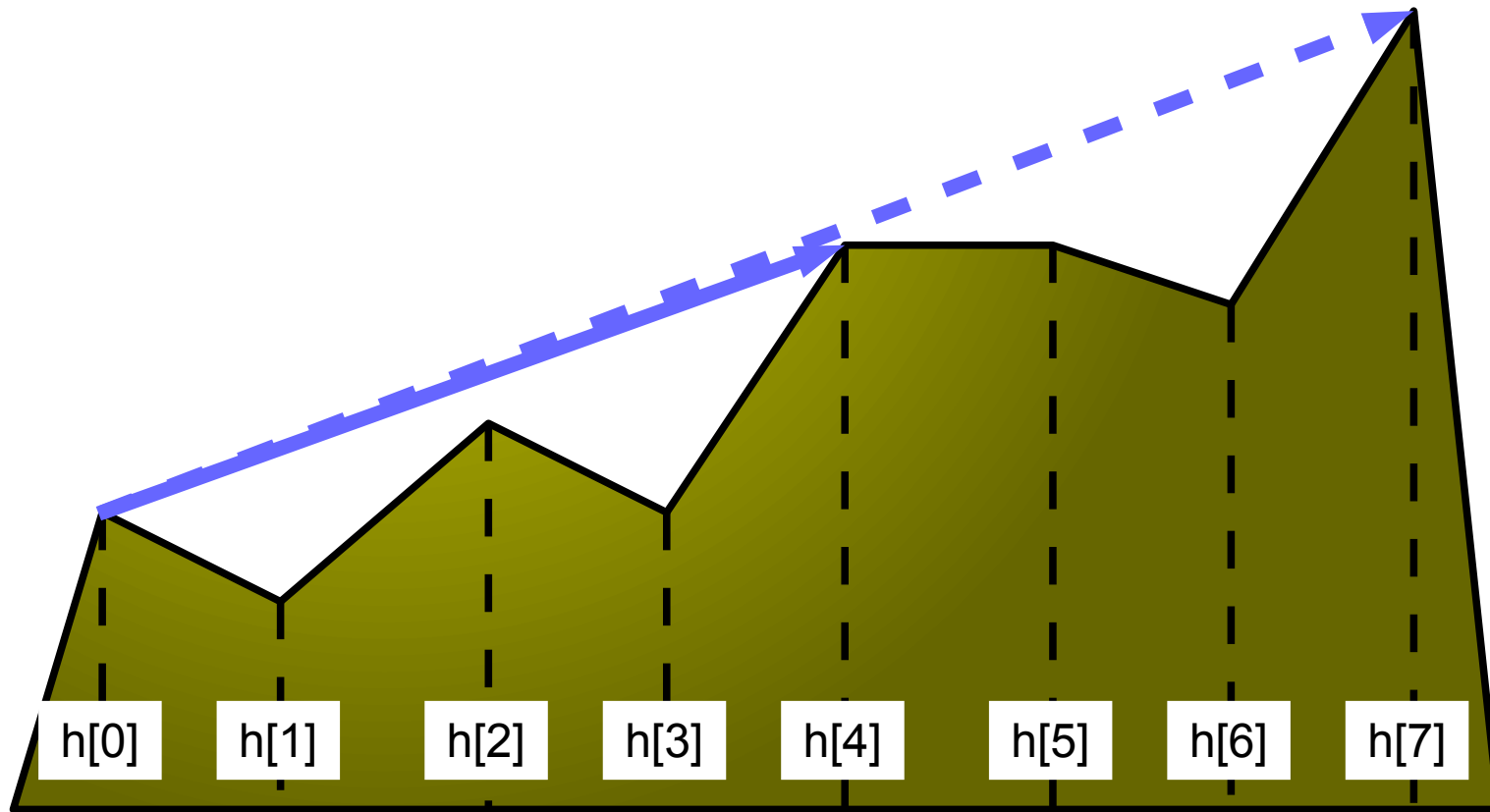
Line of sight



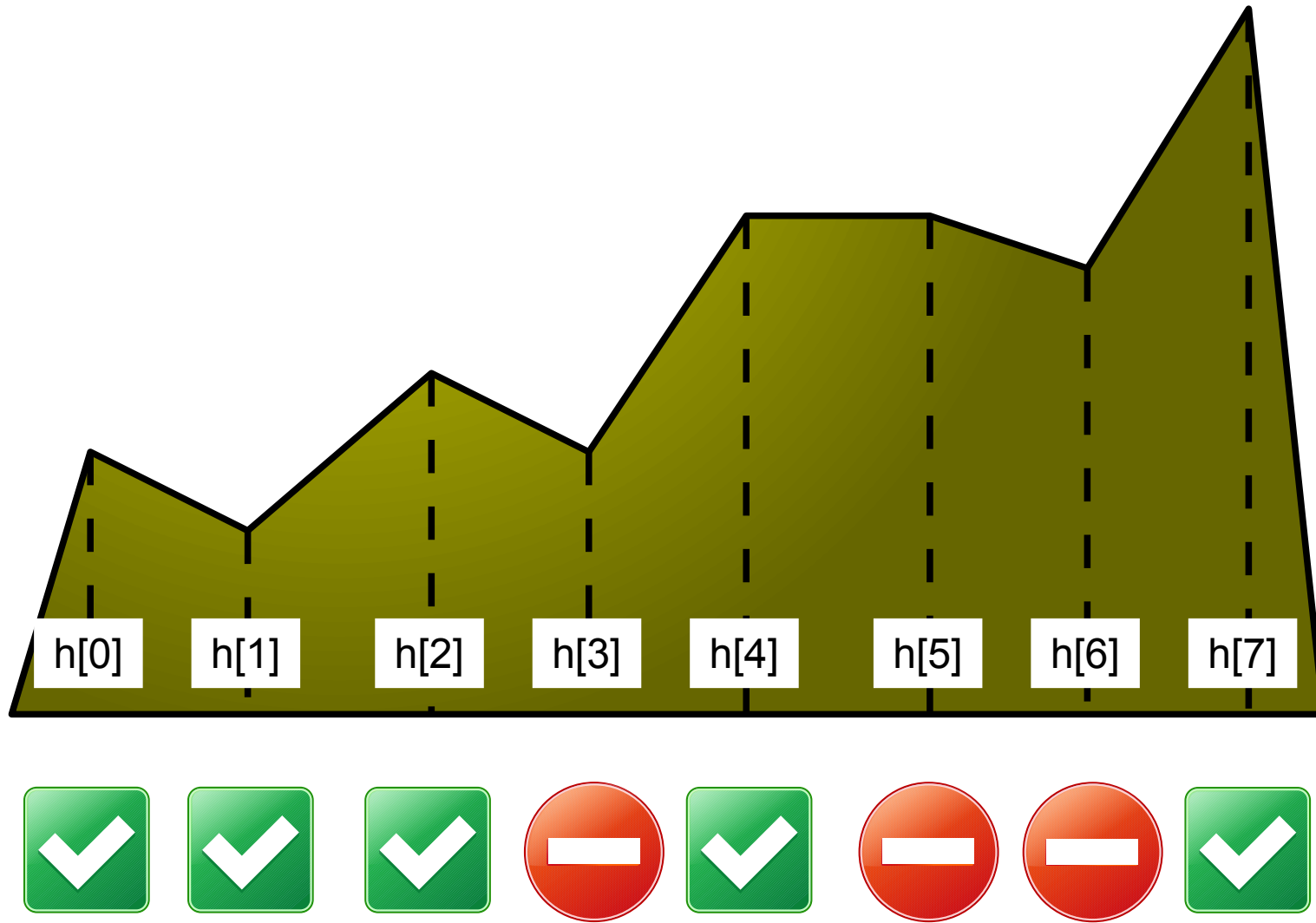
Line of sight



Line of sight



Line of sight



Serial algorithm

- For each $i = 0, \dots, n - 1$
 - Let $a[i]$ be the slope of the line connecting the peak 0 to the peak i
 - $a[0] \leftarrow -\infty$
 - $a[i] \leftarrow \arctan((h[i] - h[0]) / i),$ se $i > 0$
- For each $i = 0, \dots, n - 1$
 - $amax[0] \leftarrow -\infty$
 - $amax[i] \leftarrow \max \{a[0], a[1], \dots, a[i - 1]\},$ se $i > 0$
- For each $i = 0, \dots, n - 1$
 - If $a[i] \geq amax[i]$ then the peak i is visible
 - otherwise the peak i is not visible

Serial algorithm

```
bool[0..n-1] Line-of-sight( double h[0..n-1] )  
    bool v[0..n-1]  
    double a[0..n-1], amax[0..n-1]  
    a[0]  $\leftarrow -\infty$   
    for i  $\leftarrow$  1 to n-1 do  
        a[i]  $\leftarrow$  arctan( ( h[i] - h[0] ) / i )  
    endfor  
    amax[0]  $\leftarrow -\infty$   
    for i  $\leftarrow$  1 to n-1 do  
        amax[i]  $\leftarrow$  max{ a[i-1], amax[i-1] }  
    endfor  
    for i  $\leftarrow$  0 to n-1 do  
        v[i]  $\leftarrow$  ( a[i]  $\geq$  amax[i] )  
    endfor  
    return v
```

Serial algorithm

```
bool[0..n-1] Line-of-sight( double h[0..n-1] )
  bool v[0..n-1]
  double a[0..n-1], amax[0..n-1]
  a[0] ← -∞
  for i ← 1 to n-1 do
    a[i] ← arctan( ( h[i] - h[0] ) / i )
  endfor
  amax[0] ← -∞
  for i ← 1 to n-1 do
    amax[i] ← max{ a[i-1], amax[i-1] }
  endfor
  for i ← 0 to n-1 do
    v[i] ← ( a[i] ≥ amax[i] )
  endfor
  return v
```

Embarassingly
parallel

Not embarrassingly parallel as it uses
amax while modifying it at each step
so it is rather an exclusive scan

Embarassingly
parallel

Parallel algorithm

```
bool[0..n-1] Parallel-line-of-sight( double h[0..n-1] )
    bool v[0..n-1]
    double a[0..n-1], amax[0..n-1]
    a[0] ← -∞
    for i ← 1 to n-1 do in parallel
        a[i] ← arctan( ( h[i] - h[0] ) / i )
    endfor

    amax ← exclusive-scan( "max", a )

    for i ← 0 to n-1 do in parallel
        v[i] ← ( a[i] ≥ amax[i] )
    endfor
    return v
```

Conclusions

- A parallel programming patterns defines:
 - a partitioning of the input data
 - a communication structure among parallel tasks
- Parallel programming patterns can help to define efficient algorithms
 - Many problems can be solved by applying one or more known patterns